Introduction to Data Structures (2BTCO301) — Complete Question Bank with Model Answers

Dr. Subhash University · School of Engineering & Technology · B.Tech CSE (AI-ML) · Semester 3 Subject: Introduction to Data Structures · Code: **2BTCO301** · Source: the five unit PDFs in the `ds` Drive folder (`1hD_xgcVixNm6Lrt_kPQKEenpzVcpR8iU`) · **138 pages of university notes, Units 1–5.**

Contents

§	What is in it
0	Read this first — what the folder does and does not contain
1	The syllabus, exactly as the notes present it, with page share per unit
2	Exam pattern, marking scheme, and the 25 questions the notes practically shout
3	**Unit 1** — Introduction to Data Structures: questions and full answers
4	**Unit 2** — Arrays, Stack, Queue, Linked Lists: questions and full answers
5	**Unit 3** — Trees: questions and full answers
6	**Unit 4** — Hashing: questions and full answers
7	**Unit 5** — Sorting & Searching: questions and full answers
8	Appendix — the 21 C programs and how every number was verified
9	Revision sheet — formulas, complexity tables, the numbers to memorise, ten mark-losing mistakes
10	Errata in the notes themselves (13 items)
11	How to use this key in the last week

0. Read this first — what the folder does and does not contain

You asked for a previous-year-question analysis. For this subject that is **not possible from the material you sent**, and it would be wrong to pretend otherwise, so here is the exact inventory of that Drive folder:

File in the folder	What it is	Pages
`DS Unit-1 pdf`	Unit 1 notes — Introduction to Data Structure	12
`DS Unit-2 pdf`	Unit 2 notes — Arrays, Stack, Queue, Linked Lists	47
`DS Unit-3 pdf`	Unit 3 notes — Trees	15
`DS Unit-4 pdf`	Unit 4 notes — Hashing	6
`DS UNIT-5 pdf`	Unit 5 notes — Sorting and Searching (handwritten)	58

That is **five files, all lecture notes**. There is **no question paper, no mark scheme, no question paper photo, no assignment sheet** in this folder — unlike the DBMS, OOP and Digital Fundamentals folders, which each contained real Winter-2024 / Winter-2025 papers that could be mined for repeats.

So this book does the next best thing, and does it properly:

1. It turns **all 138 pages of the notes into exam questions** — every definition, every worked example, every table, every program, every diagram caption in the notes is converted into the question the university would ask about it (§3–§7).
2. Every answer is **worked, not recited**, and every number in every answer was **produced by executing a program** (`ds/verify_ds.py`, raw log `ds/verification_ds.txt`, 557 lines, zero checks reporting False). Where the notes print a result, the program re-runs the example and either confirms it or flags the disagreement (§10 lists all thirteen).
3. It uses the **exam pattern the university actually uses for Semester-3 CSE papers** (§2). That pattern is taken from the three real papers in your other folders; it is *not* confirmed by a DS paper, because none exists here. Plan for it, but keep 20 marks of slack in your revision budget.

If you can get hold of an actual 2BTCO301 paper — a photo from a senior, the department notice board, the university question-paper archive — send it and the same treatment (§1–§2 mapping, repeat questions marked, per-question model answers) will slot straight into this book.

How to use the rest of this book

Time you have	Read
1 day	§9 revision sheet, then §2.4 "the 25 questions the notes practically shout"
3 days	§2 → §9 → then Unit 5 (§7) and Unit 2 (§4) answers, which are ~76% of the weight
1 week+	everything in order, hand-tracing every example beside §4.4, §6.6, §7.2–§7.7

Mark scheme used throughout: **Q.1 = 10 × 2 = 20 (no choice), Q.2–Q.5 = 4 + 6 + 10 with an internal OR on the 10-mark part.** A "4-mark" answer here is 4–8 lines or one small table. A "6-mark" answer is a definition + diagram + one worked example. A "10-mark" answer is a program or a full worked trace — never prose.

---

1. The syllabus, exactly as the notes present it

Unit	Title in the notes	Pages	Share of the notes	What it actually covers
1	Introduction to Data Structure	12	8.7%	data management concepts, data types, linear vs non-linear, performance analysis, the 9 operations
2	Arrays, Stack, Queue, Linked List	47	34.1%	array ops, sparse matrices, stack + infix/postfix/prefix, recursion, queue (linear/circular/deque/priority), dynamic memory, singly/doubly/circular linked lists
3	Trees	15	10.9%	tree terminology, binary trees and their types, array/linked representation, the three traversals, BST
4	Hashing	6	4.3%	hash table model, division / mid-square / folding, collisions, linear probing, quadratic probing, double hashing, chaining, rehashing
5	Sorting and Searching	58	42.0%	internal vs external, bubble, selection, insertion, merge, quick, radix/bucket, linear search, binary search
	Total	**138**		

Two things fall straight out of that table:

- **Unit 5 alone is 42% of the material**, and Unit 2 is 34%. Together they are three quarters of the course. If you are short of time, sorting/searching and the Unit 2 data structures are where the marks live.

- **Unit 4 is only 6 pages.** It is still a guaranteed question (hashing is too convenient to leave out), but it can only support a 4- or 6-mark part, not a 10-marker. Do not over-invest there.

The notes also carry a silent asymmetry worth knowing: **Unit 5 is entirely handwritten** and is the only unit where the printed page headings and the code underneath them disagree (§10). It is the unit most likely to surprise you in the exam, so §7 traces every one of its algorithms rather than quoting the notes.

---

2. Exam pattern, marking, and what will be asked

2.1 The pattern (from the three real Semester-3 papers in your other folders)

```
Q.1   10 short questions x 2 marks                    = 20   (NO choice)
Q.2    4 + 6 + 10   with internal OR on the 10-mark     = 20
Q.3    4 + 6 + 10   with internal OR on the 10-mark     = 20
Q.4    4 + 6 + 10   with internal OR on the 10-mark     = 20
Q.5    4 + 6 + 10   with internal OR on the 10-mark     = 20
                                                Total = 100
```

Consequences:

- **Q.1 is 20 marks with no choice.** Twenty definitions × 1 minute each is the highest-yield hour of revision you can spend on this subject. §9 is written as a drill sheet for exactly that.
- **Every 10-mark part is a "do this" question** — write the program, trace the passes, draw the tree, fill the table. A 10-mark answer in DS is *working*, not explanation. The engine output in §4.4, §6.6 and §7.2–§7.7 shows what a full-marks trace looks like line by line.
- The internal OR means **you need 5 of the 10 long answers, not 10**. Prepare the two 10-mark options per unit and you are covered.

2.2 Expected unit → question placement

Question slot	Most probable unit	Why
Q.1 (shorts)	spread over 1, 2, 3, 4, 5	definitions are unit-1 and unit-3 heavy, counts are unit-2/4/5
Q.2–Q.5, 4-mark parts	Unit 1 & Unit 4	short theory, cheap to mark
6-mark parts	Unit 2 (stack/queue/linked list) & Unit 3 (trees)	definition + diagram + one example
10-mark parts	Unit 2 (conversion algorithm / linked-list program) and Unit 5 (any sort + pass table)	the only two units with enough substance for 10 marks

2.3 How marks are actually awarded in this subject

Marks	What the checker looks for
2	the exact definition or the exact term; one sentence is enough
4	definition + 2 properties **or** one small table (e.g. array vs linked list)
6	definition + labelled diagram + one worked example with the final answer
10	algorithm/program + a step-by-step trace of a given input + complexity (best/average/worst)

Never leave a 10-marker as a wall of text: draw the structure, show the passes in a table, and end with the complexity line. That is three separate mark points you collect for free.

2.4 The 25 questions the notes practically shout

Ranked by how directly the notes hand you the question — these are the ones to be able to write in your sleep:

#	Question	Unit	Where in the notes
1	Define data structure; classify primitive / non-primitive	1	p1–3
2	Linear vs non-linear data structures with examples	1	p4
3	Define performance analysis; time vs space complexity	1	p5–8
4	Best / average / worst case with an example	1	p8
5	The operations on data structures (9 of them)	1	p9–12
6	Declare / initialise / traverse a 1-D and 2-D array in C	2	p1–8
7	Advantages and disadvantages of arrays	2	p6
8	Sparse matrix: definition and triplet representation	2	p9–11
9	Stack: definition, operations, overflow and underflow	2	p12–16
10	Infix → postfix algorithm, with `A*(B+C)-D/E` traced	2	p17–20
11	Infix → prefix using the three-step method	2	p21–22
12	Postfix evaluation, e.g. `9 3 4 * 8 + 4 / -`	2	p20
13	Recursion: definition, conditions, Tower of Hanoi	2	p23–26
14	Queue: definition, operations, applications	2	p27–28
15	Circular queue: why it is needed, full/empty conditions	2	p29–31
16	Deque and priority queue with their restrictions	2	p32–34
17	Dynamic memory allocation: `malloc`, `calloc`, `realloc`, `free`	2	p35–37
18	Singly linked list: node structure + insert + delete program	2	p38–43
19	Doubly linked list and circular linked list	2	p44–46
20	Array vs linked list comparison table	2	p46–47
21	Tree terminology: node, degree, level, height, depth, leaf	3	p1–4
22	Binary tree types: full, perfect, complete, balanced, degenerate	3	p5–7
23	Preorder / inorder / postorder on a drawn tree	3	p8–14
24	Binary Search Tree: insert and search	3	p15
25	Hashing: hash function, collision, division / mid-square / folding	4	p1–6

Plus, from Unit 5: **one pass table for each of bubble, selection, insertion, merge, quick, radix**, and **linear vs binary search with the comparison count**. That is 6 more near-certainties, and all of them are traced in §7.

---

3. Unit 1 — Introduction to Data Structure (questions and full answers)

3.1 [2] Define a data structure. What does a data structure specification include?

Answer. A data structure is a representation of the logical relationship existing between individual elements of data — a way of organising data items that considers not only the elements stored but also their relationship to each other. It is a mathematical or logical model of a particular organisation of data.

The notes insist on four things a data structure specifies, and on the two memory words:

Term	Definition (memorise verbatim)
—	**Organization of data** — how the items are arranged
—	**Accessing methods** — how an item is reached
—	**Degree of associativity** — how strongly items relate to each other
—	**Processing alternatives for information** — what can be done to them
storage structure	the representation of a data structure **in main memory**
file structure	the representation of a data structure **in auxiliary memory**

The line the notes close the section with, and which is worth writing in any answer:

```
Algorithm + Data Structure = Program
```

A data-structure study answers four questions: how much memory the storage needs, how much time the processing needs, how the data is represented in memory, and which operations are performed on it.

3.2 [2] Primitive vs non-primitive data types

Answer. Primitive types are basic and are directly operated upon by machine instructions; they have different representations on different computers, and are built into most languages. Examples with the notes' own definitions:

Primitive type	Holds
`int`	values without a fractional part — whole numbers
`float`	fractional numbers
`char`	a character value
pointer	the memory address of another variable

Non-primitive types are more sophisticated, are **derived from** primitive types, and emphasise the structuring of a group of homogeneous or heterogeneous data items. The three the notes name are:

- **Array** — a fixed-size sequenced collection of elements of the same data type.
- **List** — an ordered set containing a variable number of elements.
- **File** — a collection of logically related information; a large list of records with fields.

Non-primitive further divides into **linear** and **non-linear** — which is the next question.

3.3 [4] Linear vs non-linear data structures

Answer. Linear: elements are connected in a linear fashion, logically or in sequence memory locations. Non-linear: data items are *not* arranged in a sequence.

The notes give a five-row comparison table. It is a complete 4-mark answer on its own:

Linear data structure	Non-linear data structure
Every item is related to its previous and next item; data is arranged in a linear sequence	Every item is attached with many other items; data is not arranged in sequence
Data items can be traversed in a single run	Data cannot be traversed in a single run
Examples: array, stack, linked list, queue	Examples: tree, graph
Implementation is easy	Implementation is difficult

Add the sentence that earns the last mark: a linear structure may be represented by **static** memory allocation (an array) or **dynamic** memory allocation (a linked list), and the operations possible on linear structures are traversal, insertion, deletion, searching, sorting and merging.

A tree is the notes' own bridge to Unit 3: a finite set of nodes arranged in branches and sub-branches, representing a **hierarchical** relationship, nodes drawn as circles joined by edges. A graph is nodes (information) plus edges (logical relation), and *"a tree can be viewed as a restricted graph"*.

3.4 [2] Define algorithm, and name the two complexity measures

Answer. An algorithm is a procedure you can write as a C function or program; it states explicitly how the data will be manipulated. Data structures are implemented using algorithms, which is why efficiency matters: the complexity of an algorithm is a function describing its efficiency in terms of the amount of data it must process.

Measure	Definition
Time complexity	a function describing the amount of *time* an algorithm takes in terms of the amount of input. "Time" is a natural unit — number of comparisons, number of memory accesses, number of inner-loop executions
Space complexity	a function describing the amount of *memory* an algorithm takes in terms of the input; usually only the **extra** memory, not the input's own storage

The notes add why space is often skipped: the space used is minimal or obvious — *but sometimes it becomes as important an issue as time* (that sentence is the answer to "why study space complexity").

3.5 [4] Worst case, average case and best case analysis — with linear search as the example

Answer. Three definitions to write exactly, then one worked example.

Case	What it does	Linear search (`x` compared against `a[0..n-1]`)
Worst	calculates the **upper bound** on running time; you must know the input that causes the *maximum* number of operations	`x` is not present → all `n` elements are compared → **O(n)**
Average	takes all possible inputs, computes the time for each, sums them and divides by the number of inputs; needs a (predicted) distribution	assuming all positions (including "absent") are equally likely, the sum is divided by `(n+1)` → **O(n/2) = O(n)**
Best	calculates the **lower bound**; the input causing the *minimum* operations	`x` is at `a[0]` → 1 comparison → **O(1)**, constant, independent of `n`

Average case of linear search is not a guess — it was measured. Running the search for every element of an `n`-element array and averaging the comparisons (`ds/verification_ds.txt`, "linear search vs binary search, counted"):

```
n=  8  linear worst   8 (=n)  linear avg 4.5 (= (n+1)/2 = 4.5 : True)
n= 16  linear worst  16 (=n)  linear avg 8.5 (= (n+1)/2 = 8.5 : True)
n=100  linear worst 100 (=n)  linear avg 50.5 (= (n+1)/2 = 50.5 : True)
```

The measured average equals `(n+1)/2` exactly, which is the formula to quote.

3.6 [4] List and explain the operations performed on data structures

Answer. Nine operations, in the notes' order, each with the notes' definition:

#	Operation	Definition
1	**Create**	reserves memory for program elements, by a declaration statement; may happen at compile-time or run-time. `malloc()` in C is used for creation
2	**Destroy**	destroys the memory space allocated for the data structure. `free()` in C is used
3	**Selection**	accessing a particular datum within a data structure
4	**Updation**	updates or modifies the data in the data structure
5	**Searching**	finds the presence of a desired item in the list of items; may also find the locations of all elements satisfying a condition
6	**Sorting**	arranging all data items in a particular order — ascending or descending
7	**Merging**	combining the data items of **two different sorted lists** into a single sorted list
8	**Splitting**	partitioning a single list into multiple lists
9	**Traversal**	visiting each and every node of a list in a systematic manner

Note the exact wording of 7 and 9 — they reappear as Unit 5 (`merge_two.c`) and Unit 3 (the three traversals). Two more terms the notes use, worth one line each:

- **Insertion** — adding an element to the structure.
- **Deletion** — removing an element from the structure.

3.7 [2] "Stack is a linear data structure" — justify in two lines, and give the two conditions

Answer. Insertion and deletion are performed at one end only, called the **top**, which makes it LIFO (last-in, first-out). The insertion is **PUSH**, the deletion is **POP**. Every stack has a `top` variable holding the position of the topmost element and a `MAX` for capacity; therefore

```
top = -1 (or NULL in the notes' wording)  ->  the stack is empty
top = MAX - 1                             ->  the stack is full  (push now gives overflow)
```

3.8 [2] Define a queue and name its two ends

Answer. A FIFO structure: the element inserted first is the first taken out. Insertion happens at the **rear** end and deletion at the **front** end. It may be implemented with an array or a linked list, and every queue keeps a `front` and a `rear` pointer to the positions where deletion and insertion are legal.

3.9 [6] Show that the growth of the complexities is real, not just a claim

Answer. Same input size, measured work (`ds/verification_ds.txt`, "complexity vocabulary check"), at `n = 32`:

Class	f(32)	Meaning
O(1)	1	constant — one comparison regardless of `n`
O(log n)	5.0	doubling `n` adds one step
O(n)	32	linear
O(n log n)	160.0	the sort you want
$O(n^2)$	1,024	10× more work than n log n here
$O(n^3)$	32,768	
$O(2^n)$	4,294,967,296	infeasible
O(n!)	2.6×10^{37}	not a program, a threat

The practical sentence to finish with: **binary search on 1,000,000 items needs at most 20 comparisons** (`⌊log₂ 10⁶⌋ + 1 = 20`), while linear search may need all 1,000,000.

---

4. Unit 2 — Arrays, Sparse Matrices, Stack, Expressions, Recursion

4.1 [2] How can an array be initialised? Give the three ways the notes list

Answer. An array is a collection of similar data elements stored in consecutive memory locations and referenced by an index; in C it is declared as `type name[size];`, e.g. `int marks[10];`. The three initialisation methods, with the notes' own examples:

Method	Syntax / example
At declaration	`type array_name[size] = {list of values};` → `int marks[5] = {90, 82, 78, 95, 88};` (values inside braces, comma separated)
From the keyboard	a `for` / `while` / `do-while` loop that reads one value per element
Element by element	the assignment operator: `marks[3] = 100;`

4.2 [4] Types of arrays — syntax for each

Answer.

Type	Syntax in the notes	Example
One dimensional	`data_type array_name[size];`	`int a[10];`
Two dimensional	`data_type array_name[row-size][column-size];`	`int marks[3][5];`
Multi dimensional	`data_type array_name[I1][I2][I3]…[In];`	`int x[2][3][4];`

The four operations on arrays the notes name are **traversing, inserting, searching, deleting**. Insert at the end is two steps:

```
Step 1: set upper_bound = upper_bound + 1
Step 2: A[upper_bound] = VAL
Step 3: EXIT
```

Delete at the end is one step: `SET upper_bound = upper_bound - 1`. Insertion/deletion *in the middle* needs the shift loop — and that shift is exactly why "insertion and deletion in an array is quite tricky, as the array stores elements in continuous form" (one of the notes' disadvantages).

4.3 [4] Advantages and disadvantages of arrays

Answer. Both lists come straight from the notes; write five and four.

Advantages:

1. store multiple elements of the same type under one name;
2. random access using an index number;
3. memory is predefined, so there is no extra memory loss;
4. arrays avoid memory overflow;
5. 2-D arrays efficiently represent tabular data.

Disadvantages:

1. the number of elements must be predefined;
2. an array is **static** — it cannot alter its size after declaration;
3. insertion/deletion is tricky because the elements are stored continuously (everything after the gap must shift);
4. allocating more than required wastes memory.

4.4 [4] Address arithmetic: locate `A[3][4]` of `int A[10][8]` with base 1000 and 2 bytes per element

Answer. Row-major order stores row after row, so

```
LOC(i, j) = BASE + ( (i - LB1) * N + (j - LB2) ) * W          N = number of columns
```

With `LB1 = LB2 = 1`, `N = 8`, `BASE = 1000`, `W = 2`:

```
LOC(3, 4) = 1000 + ((3-1)*8 + (4-1)) * 2 = 1000 + (16 + 3) * 2 = 1038
LOC(1, 1) = 1000
LOC(10, 8)= 1000 + ((10-1)*8 + (8-1)) * 2 = 1000 + 79*2 = 1158
```

The formula was checked against a brute-force enumeration of all 80 cells — it agrees on every one (`ds/verification_ds.txt`, "array address arithmetic (row-major)"). Column-major swaps the roles of row and column:

```
LOC_col(i, j) = BASE + ((j - LB2) * nrows + (i - LB1)) * W  ->  LOC_col(3,4) = 1064
```

The same arithmetic for a 3×4 array printed next to the values it addresses (`ds/c/array_2d.c`):

```
   LOC(i,j) = 1000 + ((i-1)*4 + (j-1)) * 2   for i=1..3, j=1..4
      LOC(1,1)=1000  value 1     LOC(1,2)=1002  value 2     LOC(1,3)=1004  value 3     LOC(1,4)=1006  value 4
      LOC(2,1)=1008  value 5     LOC(2,2)=1010  value 6     LOC(2,3)=1012  value 7     LOC(2,4)=1014  value 8
      LOC(3,1)=1016  value 9     LOC(3,2)=1018  value 10    LOC(3,3)=1020  value 11    LOC(3,4)=1022  value
12
```

4.5 [6] Sparse matrix — definition, the three types, and the storage saving

Answer. A sparse matrix is a matrix that has a large number of zero-valued elements. Storing all of those zeros is waste, so only the non-zeros are kept, usually in a **triplet** form: three rows — `row index`, `column index`,

`value` — with a final column holding `rows, cols, non-zero count`.

The three types, with the notes' zero-conditions:

Type	Zero condition	Non-zero count for an `n × n` matrix
Lower-triangular	`A[i][j] = 0` where `i < j`	`n(n+1)/2`
Upper-triangular	`A[i][j] = 0` where `i > j`	`n(n+1)/2`
Tri-diagonal	`A[i][j] = 0` where `|i − j| > 1`	`3n − 2`

The upper-triangular description in the notes — *n non-zeros in the first row, n−1 in the second, down to one in the nth row* — sums to `n(n+1)/2`, which is the same count as the lower one by symmetry. The tri-diagonal count is justified the same way: `n` on the main diagonal (`i = j`) + `n−1` below (`i = j+1`) + `n−1` above (`i = j−1`) = `3n − 2`.

Every one of those formulas was verified by *enumerating* the qualifying cells, not by algebra (`ds/verification_ds.txt`, "sparse matrices: non-zero counts"):

```
n=1  lower n(n+1)/2= 1 (enumerated  1:True)  upper= 1:True  tridiagonal 3n-2= 1:True
n=2  lower n(n+1)/2= 3 (enumerated  3:True)  upper= 3:True  tridiagonal 3n-2= 4:True
n=3  lower n(n+1)/2= 6 (enumerated  6:True)  upper= 6:True  tridiagonal 3n-2= 7:True
n=4  lower n(n+1)/2=10 (enumerated 10:True)  upper=10:True  tridiagonal 3n-2=10:True
n=5  lower n(n+1)/2=15 (enumerated 15:True)  upper=15:True  tridiagonal 3n-2=13:True
n=6  lower n(n+1)/2=21 (enumerated 21:True)  upper=21:True  tridiagonal 3n-2=16:True
n=7  lower n(n+1)/2=28 (enumerated 28:True)  upper=28:True  tridiagonal 3n-2=19:True
n=100: full storage 10000 cells vs lower-triangular 5050 vs tridiagonal 298 -> saving 49.5%
```

Two exam-useful readings from that table: at `n = 4` the tri-diagonal count `3n−2 = 10` **equals** the triangular count `n(n+1)/2 = 10` — below `n = 4` tri-diagonal is *worse*, so the compact form only pays off from `n ≥ 5`. And at `n = 100`, a lower-triangular matrix still needs 5050 of 10000 cells (49.5% saved), whereas a genuinely sparse matrix with 4 non-zeros in a 4×4 needs only `3 × (4+1) = 15` of 16 — the real win comes from the *number of non-zeros*, not from the shape.

Worked triplet example, run as `ds/c/sparse_matrix.c`:

```
4x4 matrix, 16 cells, 4 non-zero
   row :   0   1   2   3     4
   col :   2   1   3   0     4
   val :   5   8   3   9     4
stored cells = 3*(4+1) = 15 instead of 16
```

4.6 [4] Stack: definition, the three operations, overflow and underflow

Answer. A linear list which allows insertion and deletion at one end only, called the **top**; insertion is **PUSH**, deletion is **POP**, and it is therefore a **LIFO** list. A pointer `TOP` keeps track of the top element: initially `TOP = null` or `-1` for an empty stack; `TOP` is incremented *before* an element is placed, and decremented on each deletion. With capacity `MAX`:

```
TOP == MAX - 1   ->  the stack is full   ->  push gives OVERFLOW
TOP == -1        ->  the stack is empty  ->  pop  gives UNDERFLOW
```

The three operations named in the notes are `push()`, `pop()` and `PEEP(S, TOP, I)` — and the note on `PEEP` is the mark: *it returns the value of the I-th element from the TOP; the element is **not** deleted by this function.*

Program and its executed output (`ds/c/stack_array.c`, MAX = 5):

```
push 10 -> ok, top=10
push 20 -> ok, top=20
push 30 -> ok, top=30
push 40 -> ok, top=40
push 50 -> ok, top=50
push 60 -> Overflow
peep  -> 50
pop   -> 50
pop   -> 40
pop   -> 30
pop   -> 20
pop   -> 10
pop   -> Underflow
pop   -> Underflow
peep  -> the stack is empty
```

Four applications to list (the notes give exactly these): conversion of infix to prefix, evaluation of a prefix expression, recursion, and Tower of Hanoi.

4.7 [4] Infix, postfix and prefix — define each with an example and the precedence rules

Answer.

Notation	Form	Example	Also called
Infix	Operand Operator Operand	`A + B`	—
Postfix	Operand Operand Operator	`AB+`	**reverse Polish notation**
Prefix	Operator Operand Operand	`+AB`	**Polish notation**

Precedence (the notes' wording): **higher priority** `* / %`, **lower priority** `+ -`; the order of evaluation can be changed with parentheses — in `A+B*C` the `B*C` is done first and the result added to `A`, while `(A+B)*C` evaluates `A+B` first.

Why bother converting: postfix/prefix need **no parentheses and no precedence rules**, so a machine can evaluate them in one left-to-right scan with a stack.

4.8 [10] Convert `A*(B+C)-D/E` into postfix, stage by stage, using the stack algorithm

Answer. The notes walk 13 stages. The rule set that produces them:

```
operand           -> append to output immediately
'('               -> push  (highest priority going in, lowest coming out)
')'               -> pop to output until '(' is met; discard both
operator          -> pop to output every stack-top of >= equal precedence
                     (left-associative), then push
end of input      -> pop everything left on the stack
```

Stage	Token	Action	Output so far	Stack
1	—	empty stack	*(nothing)*	—
2	`A`	operand, append	`A`	—
3	`*`	stack empty → push	`A`	`*`
4	`(`	push; precedence of `(` is maximum going in, least once something sits on it	`A`	`* (`
5	`B`	operand	`A B`	`* (`
6	`+`	top is `(`, outgoing precedence of `(` is least → push	`A B`	`* ( +`
7	`C`	operand	`A B C`	`* ( +`
8	`)`	pop until `(`	`A B C +`	`*`
9	`-`	top `*` has higher precedence → pop `*`, then push `-`	`A B C + *`	`-`
10	`D`	operand	`A B C + * D`	`-`
11	`/`	`/` outranks `-` on the stack → push	`A B C + * D`	`- /`
12	`E`	operand	`A B C + * D E`	`- /`
13	end	pop the stack	`A B C + * D E / -`	—

Answer: `ABC+*DE/-` — confirmed twice: by the shunting-yard implementation in `ds/c/infix_postfix.c`, and independently by `ds/verify_ds.py`:

```
A*(B+C)-D/E          -> postfix ABC+*DE/-        notes: ABC+*DE/-        match: True
```

Also reproduced from the notes on the same program:

```
(A-B)*(C+D)          -> postfix AB-CD+*          notes: AB-CD+*          match: True
(A+B)/(C+D)-(D*E)    -> postfix AB+CD+/DE*-      notes: AB+CD+/DE*-      match: True
```

4.9 [6] Evaluate the postfix expression `9 3 4 * 8 + 4 / -`

Answer. Rule: scan left to right; push operands; on an operator pop the **top two** values, apply, push the result. The order matters — the first value popped is the **right** operand.

Token	Action	Stack after
`9`	push	`9`
`3`	push	`9 3`
`4`	push	`9 3 4`
`*`	pop 4 (right), pop 3 (left) → 3 × 4 = 12	`9 12`
`8`	push	`9 12 8`
`+`	pop 8, pop 12 → 12 + 8 = 20	`9 20`
`4`	push	`9 20 4`
`/`	pop 4, pop 20 → 20 / 4 = 5	`9 5`
`-`	pop 5, pop 9 → 9 − 5 = 4	`4`

Executed trace (the row order above is the scan order; the program prints it exactly):

```
    3 * 4 = 12   stack now [9, 12]
    12 + 8 = 20   stack now [9, 20]
    20 / 4 = 5   stack now [9, 5]
    9 - 5 = 4   stack now [4]
    value = 4 | python check 9 - ((3*4+8)/4) = 4.0 | match: True
```

Answer: 4, and the notes' matching infix form is `9 - ((3 * 4) + 8) / 4`.

4.10 [6] Convert infix to prefix by the three-step method: `(A - B/C) * (A/K - L)`

Answer. The notes' method — and it is the one to write, because it reuses the postfix algorithm:

```
Step 1  reverse the whole infix string, interchanging '(' with ')'
Step 2  find the POSTFIX of that reversed expression
Step 3  reverse that postfix -> it is the PREFIX of the original
```

Step	Result
Step 1, reverse + swap brackets	`(L - K/A) * (C/B - A)`
Step 2, postfix of that	`L K A / - C B / A - *`
Step 3, reverse	`* - A / B C - / A K L`

Answer: `*-A/BC-/AKL`. Both the three-step method and a direct prefix algorithm were run and give the identical string (`ds/verification_ds.txt`):

```
    step1 reverse + swap brackets : (L-K/A)*(C/B-A)
    step2 postfix of the reversed : LKA/-CB/A-*
    step3 reverse -> prefix       : *-A/BC-/AKL
    direct algorithm gives        : *-A/BC-/AKL | identical: True
```

One caution, because the trick is stated too loosely in the notes. Reversing the expression also reverses the *order in which same-precedence operators are met*, so the middle pass has to treat every operator as right-associative. Followed literally with the ordinary left-associative postfix algorithm, `A-B+C` comes out as `-A+BC`, which means `A - (B + C)` — wrong, since `A - B + C` is `(A - B) + C` and its prefix is `+-ABC`. The engine handles this (`infix_to_prefix` pops only on strictly greater precedence), which is why the harder example below also comes out right:

```
A*(B+C)-D/(A*K)+L    -> postfix ABC+*DAK*/-L+  match: True
A*(B+C)-D/(A*K)+L    -> prefix  +-*A+BC/D*AKL  match: True
```

See §10, errata row 13.

The three simpler prefix examples from the notes also reproduce exactly:

```
A+B                  -> prefix  +AB            notes: +AB            match: True
(A+B)*C              -> prefix  *+ABC          notes: *+ABC          match: True
(A-B)*(C+D)          -> prefix  *-AB+CD        notes: *-AB+CD        match: True
(A+B)/(C+D)-(D*E)    -> prefix  -/+AB+CD*DE    notes: -/+AB+CD*DE    match: True
```

4.11 [4] Define recursion and state the conditions a recursive function must satisfy

Answer. A recursive function is one that calls itself to solve a smaller version of its task, until a final call is made that does not require a call to itself. Because it repeatedly calls itself, it uses the **system stack** to store the return address and local variables of the calling function — which is why recursion is listed as an application of a stack. Recursion defines large, complex problems in terms of simpler ones: `n! = n × (n−1)!`.

The conditions, in order:

1. a **base case** that stops the recursion (`if n == 0 return 1`);
2. every recursive call must move **toward** the base case (smaller sub-problem);
3. the function must be **defined** in terms of itself with a finite number of calls — otherwise the system stack grows until it overflows.

Executed check: `recursive factorial 5! = 120`, `fib(10) via recursion = 55`.

4.12 [10] Tower of Hanoi — state the problem, give the recurrence, and solve it for 3 discs

Answer. The notes: three rings on pole **A** must be moved to pole **C** while keeping the same order, using spare pole **B**; the rule is that **a smaller disc must always come above a larger one**. It is the classic recursion application: *"if you can solve n−1 cases, then you can easily solve the nth case"*.

```
move(n, from, to, spare):
    if n == 0: return
    move(n-1, from, spare, to)      # get the stack out of the way
    move disc n  from  ->  to       # the single legal move of the biggest disc
    move(n-1, spare, to, from)      # put the stack back on top
```

Recurrence: `T(n) = 2 T(n−1) + 1`, `T(1) = 1`, hence `T(n) = 2^n − 1`. Measured, not asserted:

```
n=1 discs : moves = 2^1 - 1 = 1
n=2 discs : moves = 2^2 - 1 = 3
n=3 discs : moves = 2^3 - 1 = 7
n=4 discs : moves = 2^4 - 1 = 15
n=5 discs : moves = 2^5 - 1 = 31
```

The complete 3-disc solution, printed by the recursive routine:

```
    move disc 1 from A to C
    move disc 2 from A to B
    move disc 1 from C to B
    move disc 3 from A to C
    move disc 1 from B to A
    move disc 2 from B to C
    move disc 1 from A to C
```

Two follow-up marks: the solution is **optimal** (no algorithm can do it in fewer than `2^n - 1` moves), and for `n = 64` the count `2^64 - 1 ≈ 1.8 × 10^19` is the "world ends" story — at one move per second it exceeds the age of the earth.

4.13 [4] Queue: definition, the two ends, the two operations, and the real-life reading

Answer. A queue is an ordered list in which insertion is done at one end called **REAR** and deletion at another end called **FRONT** — a FIFO structure in which the element inserted first is the first to be taken out. Every queue keeps two pointers: `front` (where deletions happen) and `rear` (where insertions happen). It can be implemented with an array or a linked list.

The notes' real-life reading, worth one line in a 4-marker: a ticket queue outside a counter, students standing in a queue for assembly prayer, a queue outside a theatre booking counter — in all of them the person who arrives first is the first served.

Operations named in the notes: **Insert** (at rear) and **Delete** (at front), plus the standard `isEmpty()` / `isFull()` tests.

4.14 [6] Why is a circular queue needed? Show the wasted space in a linear queue

Answer. Take `MAX = 10`, fill it, then delete two elements.

Moment	front	rear	What is true
after 10 enqueues	0	9	`rear == MAX-1`, the queue is genuinely full
after 2 dequeues	2	9	cells 0 and 1 are free, but `rear` has not moved
next insert	—	—	**refused**, because the linear-queue full test is still `rear == MAX-1`

Measured (`ds/verification_ds.txt`, "linear queue wasted space vs circular queue"):

```
after 10 enqueues: front=0 rear=9 (rear==MAX-1 -> full)
after 2 dequeues : front=2 rear=9 -> 2 cells free at indexes 0,1
linear queue refuses the next insert because rear==MAX-1: True (this is the drawback)
usable capacity after those deletions = 8 of 10 cells
circular queue: front=2 rear=0, the new insert landed at index 0: True
conditions - empty: front==-1 ; full: (rear+1)%MAX==front ; here (9+1)%10==0 -> True
```

The notes give exactly two remedies and reject the first: **shift every element left** (correct but "very time-consuming, especially when the queue is quite large") or **use a circular queue**, where "the first index comes right after the last index". A circular queue is the extended version of a regular queue in which the last element is connected to the first; it is also called a **buffer** or **ring buffer**, and it follows FIFO like a linear queue — the only change is the wrap-around.

The two conditions to memorise, with the modulo that makes them work:

```
empty : front == -1
full  : (rear + 1) % MAX == front
insert: rear = (rear + 1) % MAX ; Q[rear] = VAL
delete: VAL = Q[front] ; if front == rear then front = rear = -1
                         else front = (front + 1) % MAX
```

Executed run of `ds/c/queue_circular.c` with `MAX = 5` — note the wrap at `insert 70`:

```
insert 10 -> ok, front=0 rear=0
insert 20 -> ok, front=0 rear=1
insert 30 -> ok, front=0 rear=2
insert 40 -> ok, front=0 rear=3
insert 50 -> ok, front=0 rear=4
insert 60 -> Queue Overflow
delete -> 10, front=1 rear=4
delete -> 20, front=2 rear=4
delete -> 30, front=3 rear=4
insert 70 -> ok, front=3 rear=0        <-- wrapped into the cell a linear queue wastes
insert 80 -> ok, front=3 rear=1
insert 90 -> ok, front=3 rear=2
delete -> 40, front=4 rear=2
delete -> 50, front=0 rear=2
delete -> 70, front=1 rear=2
delete -> 80, front=2 rear=2
delete -> 90, front=-1 rear=-1
delete -> Queue Underflow
```

4.15 [4] Deque — definition and its two variants

Answer. A **deque** (double-ended queue) is a list in which elements can be inserted **or** deleted at **either end**; it is also called a **head–tail linked list**. Two pointers, `LEFT` and `RIGHT`, point to the two ends. The restriction that defines it: **no element can be added or deleted from the middle.**

Variant	Insertion allowed at	Deletion allowed at
Input-restricted deque	only one end	both ends
Output-restricted deque	both ends	only one end

One sentence to add: a queue is a special case of a deque (insert at one end, delete at the other), and a stack is another special case (insert and delete at the same end) — so a deque generalises both.

4.16 [6] Priority queue — rules, linked representation, and the notes' insertion example

Answer. A priority queue is a data structure in which **each element is assigned a priority**, and the priority decides the order in which elements are processed. The two general rules:

1. an element with higher priority is processed before an element with lower priority;
2. two elements with the same priority are processed **FCFS** (first-come-first-served).

Linked representation: each node has **three** parts — (a) the information/data part, (b) the priority number, (c) the address of the next element. If the list is kept **sorted**, the higher-priority element precedes the lower-priority one, so insertion must find the right place rather than append.

Insertion rule, verbatim from the notes: traverse until a node with **lower** priority than the new element is found and insert **before** it; if an element with the **same** priority exists, the new element is inserted **after** it (that "after" is what makes the structure FCFS among equals).

The notes' example — list `A(1) B(2) C(3) D(5)`, insert `F` with priority 4 — executed:

```
list sorted by priority (lower number = higher priority): [('A',1), ('B',2), ('C',3), ('D',5)]
F goes at index 3 (first entry whose priority number is larger than 4 is D)
insert F with priority 4 -> [('A',1), ('B',2), ('C',3), ('F',4), ('D',5)] | F placed before D(5): True
priorities in the result: [1, 2, 3, 4, 5] non-decreasing: True
```

Answer: `A(1) → B(2) → C(3) → F(4) → D(5)`, exactly as the notes claim.

Applications of queues to list (the notes give these four): waiting lists for a single shared resource — printer, disk, CPU; buffers on MP3/portable CD players and the iPod playlist (add songs at the end, play from the front); operating-system interrupt handling, where interrupts must be served in arrival order and so a FIFO queue is the appropriate structure.

4.17 [6] Dynamic memory allocation — `malloc`, `calloc`, `realloc`, `free`

Answer. An array created at compile time by specifying its size has a fixed size that cannot be modified at run time; that is **static memory allocation** and such arrays are **static arrays**. C can also allocate memory for an array **at run time** — **dynamic memory allocation**, giving **dynamic arrays**, created through pointer variables plus four memory-management functions.

Function	Syntax	Behaviour, in the notes' words
`malloc`	`ptr = (cast-type*) malloc(byte-size);`	allocates a block of the requested bytes and returns a pointer to the **first byte**; returns a **void pointer** so it can be assigned to any pointer type; returns a **null pointer if no memory is available**. `int *x; x = (int*)malloc(20);` allocates 20 bytes
`calloc`	`ptr = (cast-type*) calloc(n, element-size);`	requests memory at run time, used for derived types such as arrays and structures; **malloc allocates a single block, calloc allocates multiple blocks each of the same size**; returns null if there is not enough memory. The notes' example allocates 30 `struct student` records: `st_ptr = (record*) calloc(class_size, sizeof(record));`
`realloc`	`p_var = realloc(p_var, new_size);`	changes the size of previously `malloc` / `calloc` -allocated memory. Growing leaves the **old contents unchanged** and adds memory at the end of the block; shrinking keeps the remaining contents. The process is called **reallocation**
`free`	`free(ptr);`	returns the allocated block to the system — the **Destroy** operation of Unit 1

Two marks that students lose: `malloc` leaves the block **uninitialised**, `calloc` **zero-initialises** it; and forgetting `free` is a memory leak, because dynamic memory is not reclaimed when the pointer goes out of scope.

4.18 [4] Linked list — node structure, the basic operations, and its applications

Answer. A linked list is a very flexible, **dynamic** data structure in which elements (nodes) form a sequential list; each node is allocated space as it is added, and every node contains two kinds of data — the **value** and a **pointer/link to the next node**.

```
struct Node
{
    int data;      /* data of the node */
    Node* next;    /* pointer to the next node */
};
```

The list is represented by a pointer to the first node, the **head**; an empty list has `head == NULL`. To traverse, start at the head and follow `next` until a node whose `next` is `NULL`.

Basic operations named in the notes: **Insertion** (add at the beginning), **Deletion** (delete at the beginning), **Display** (the complete list), **Search** (by key), **Delete** (by key).

Applications, all six from the notes: line editors (a list of line nodes: line number, text, pointer to next); string manipulation (variable-length strings as a count plus a list of character nodes); operating systems (multi-user systems keep user jobs waiting to execute in a linked queue of control blocks); implementing stacks and queues (easier with lists than with arrays); LISP programming; and associative arrays, which are then called **association lists**.

4.19 [10] Singly linked list in C — insert at front, insert at end, search, delete, display

Answer. The three insertion cases the notes separate:

```
Insert at the Beginning (Head)
  1. create a new node with the given data
  2. set the new node's next to the current head
  3. update head to point to the new node

Insert at the End (Tail)
  1. create the new node with next = NULL
  2. if the list is empty, head = new node
  3. otherwise traverse to the last node and set its next = new node

Insert after a Given Node
  1. newNode->next = current->next
  2. current->next = newNode
```

Full program (compiled clean by `ds/c/sll.c`):

```
#include <stdio.h>
#include <stdlib.h>
struct node{ int data; struct node *next; };
struct node *head=NULL;
void insert_front(int v){ struct node *n=malloc(sizeof *n); n->data=v; n->next=head; head=n; }
void insert_end(int v){
    struct node *n=malloc(sizeof *n); n->data=v; n->next=NULL;
    if(!head){head=n;return;} struct node *p=head; while(p->next)p=p->next; p->next=n;
}
void show(const char *tag){
    struct node *p=head; int cnt=0;
    printf("%s: ", tag);
    for(p=head;p;p=p->next){ printf("%d->",p->data); cnt++; }
    printf("NULL   (length %d)\n", cnt);
}
int search(int v){ int i=1; for(struct node*p=head;p;p=p->next,i++) if(p->data==v) return i; return -1; }
void del_val(int v){
    struct node *p=head,*q=NULL;
    while(p && p->data!=v){ q=p; p=p->next; }
    if(!p){ printf("delete %d -> not found\n", v); return; }
    if(q) q->next=p->next; else head=p->next;
    free(p);
    printf("delete %d -> done\n", v);
}
int main(void){
    int i, in[]={20,30,40};
    for(i=0;i<3;i++) insert_end(in[i]);
    show("after 20,30,40 at the end");
    insert_front(10); show("after 10 at the front");
    printf("search 30 -> position %d\n", search(30));
    printf("search 99 -> position %d\n", search(99));
    del_val(30); show("after deleting 30");
    del_val(10); show("after deleting the head 10");
    del_val(30);
    return 0;
}
```

Output (from the executed model of the same logic):

```
after 20,30,40 at the end, then 10 at the front: 10->20->30->40->NULL   (length 4)
search 30 -> position 3
search 99 -> position -1
delete 30 -> done
   list now: 10->20->40->NULL   (length 3)
delete 10 -> done
   list now: 20->40->NULL   (length 2)
delete 30 -> not found
```

Three things to point at in the exam to get the last marks: deleting the head needs the `q == NULL` special case; every `free()` must happen after the pointers are re-linked, never before; and search is **O(n)** because there is no index — you walk the list.

4.20 [6] Doubly linked list and circular linked list — how do they differ from a singly linked list?

Answer.

- **Doubly linked list (DLL).** Each node has **three** "buckets": the data, the address of the **previous** node, and the address of the **next** node. Unlike a singly linked list, navigation is possible in both directions; the notes' example application is a music player (previous track / next track). Deletion has three primary cases (head, middle, tail), and because each node carries `prev`, deleting a node you already hold costs no traversal.
- **Circular linked list (CLL).** The last node's `next` points **back to the first node** instead of `NULL`, so there is no natural start or end; a circular list may be singly or doubly linked. The warning the notes give is the one that costs marks: *"the traversal in this linked list will go on forever until it is broken"* — so a traversal must stop when `next == head`, not when `next == NULL`. Insertion is done at the start or after a given node; for the first node, `newNode->next = newNode` (it points to itself) and `*head = newNode`.

Executed check of the backward walk, `ds/c/dll.c`:

```
appended 10,20,30,40,50; walking from the tail through prev:
   50 40 30 20 10
```

4.21 [10] Compare array with linked list, and singly with doubly linked list

Answer. Both tables are printed in the notes (Unit 2, pp. 46–47) as images; they are reproduced here in full because either one can be a 10-marker.

Difference between array and linked list

Feature	Array	Linked List
Memory Allocation	contiguous blocks of memory where elements are stored sequentially	nodes not necessarily in contiguous memory; each node contains data and a reference to the next node
Size Flexibility	typically a fixed size determined at creation	can dynamically grow or shrink by adding or removing nodes
Insertions and Deletions	inefficient, especially in the middle, as it may require shifting elements	efficient at any position without shifting — only pointers are updated
Random Access	constant-time **O(1)** access by index	requires **O(n)** — you must traverse from the beginning
Memory Overhead	lower: stores only data elements and an index	higher: data + pointers + node structures
Memory Allocation Type	on the **stack** for fixed-size arrays, or the **heap** for dynamic arrays	dynamic memory allocation for each node
Implementation Complexity	simpler and more straightforward	requires managing nodes and pointers, more complex
Cache Performance	better, because of the contiguous layout → fewer cache misses	worse, because memory locations are scattered → more cache misses
Contiguous vs Non-Contiguous	stores elements contiguously	does not require contiguous allocation
Use Cases	fast random access, size known and fixed	dynamic sizing, frequent insert/delete, size unknown or variable

Difference between singly and doubly linked list

Sr.	Key	Singly linked list	Doubly linked list
1	Complexity	insertion and deletion at a known position is **O(n)**	insertion and deletion at a known position is **O(1)**
2	Internal implementation	node contains data and a pointer to the next node	node contains data and pointers to the next **and** the previous node
3	Order of elements	traversal in one way only	allows two-way traversal
4	Usage	generally used to implement stacks	used to implement stacks as well as heaps and binary trees
5	Index performance	preferred when memory must be saved and searching is not required, since one pointer is stored	preferred when better searching performance is needed and memory is not a limitation
6	Memory consumption	stores a pointer to only one node, so consumes lesser memory	uses more memory per node (two pointers)

The sentence that ties row 1 to reality: in a singly list, reaching position k costs k steps even if you already know you want it; in a doubly list you can be handed the node itself (for example from a stored pointer) and unlink it in constant time, because its predecessor is available.

---

5. Unit 3 — Trees (questions and full answers)

5.1 [2] Define a tree and justify "tree is a non-linear data structure"

Answer. A tree is a **non-linear hierarchical** data structure comprising a collection of entities called **nodes**, connected by **edges** (both directed and undirected). It stems from a single node called the **root**, with subtrees attached to the root; it is a collection of nodes connected by edges with a hierarchical relationship between them. The topmost node is the root, the nodes below are child nodes, each node may have multiple children, and those children may have children of their own.

Why non-linear — the notes' own answer: the data in a tree are **not stored in a sequential manner**; they are arranged on **multiple levels**, i.e. in a hierarchical structure.

5.2 [6] Tree terminology — define every term the notes list

Answer. Twelve definitions, in the notes' order. Write them as a two-column table; it is faster to mark and faster to recall.

Term	Definition
Node	an entity that contains a key or value and pointers to its child nodes
Root	the topmost node; the first/initial node; **a tree has exactly one root**
Edge	the connecting link between any two nodes; **N nodes are connected by N − 1 edges**
Parent	the node that is the predecessor of a node — a node with a branch from itself to a successive node
Child	a descendant of a node; **every node except the root is a child node**
Siblings	nodes that belong to the same parent
Leaf	a node with no child; also called an **external** or **terminal** node
Internal node	a node with at least one child; every node other than a leaf; the root counts as internal when the tree has more than one node
Degree of a node	the total number of children of that node
Degree of the tree	the **highest** node degree among all nodes
Level	root at level 0, its children at level 1, their children at level 2, and so on
Height	the number of edges on the longest path from the node down to a leaf; the height of the root is the **height of the tree**; every leaf has height 0
Depth	the number of edges from the root to that node; the depth of the root is 0
Subtree	each child of a node forms a subtree recursively; every child is the root of a subtree of its parent

The height/depth pair is the one place the notes are loose with wording — see §9.2. Use the definitions above (height = down to the deepest leaf, depth = up to the root); they are the ones that make the numerical facts consistent.

The identity `N nodes → N − 1 edges` is not just quoted here, it was checked on all three example trees in §5.5: 13 nodes/12 edges, 12 nodes/11 edges, 9 nodes/8 edges — all `True`.

5.3 [4] General tree vs binary tree

Answer. A **general tree** places **no constraints** on the hierarchical structure — a node may have any number of children. A **binary tree** has these properties:

1. it follows all properties of a tree;
2. it can have **at most two** child nodes;
3. those two children are named the **left child** and the **right child**.

So a binary tree is a general tree whose root holds at most two subtrees — a left subtree and a right subtree. Each node of a binary tree consists of **three items**: the data item, the address of the left child, and the address of the right child. Applications the notes name: expression parsing, binary search trees, and heaps.

5.4 [6] Types of binary tree

Answer. Six types, exactly as the notes classify them, each with the test to apply.

Type	Definition	Quick test
Full	every parent/internal node has **either two or no** children	no node has exactly one child
Perfect	every internal node has **exactly two** children **and all leaves are at the same level**; all internal nodes have degree 2	`n = 2^(h+1) - 1`
Complete	all levels are completely filled **except possibly the lowest, which is filled from the left**	leaf elements lean left; the last leaf may have no right sibling
Degenerate / pathological	every node has a **single** child, either left or right	it is a linked list wearing a tree costume
Balanced (height-balanced)	for **every** node the heights of the left and right subtrees differ by **not more than 1** — plus the left subtree is balanced and the right subtree is balanced	check all three clauses, not just the first
Strictly binary	every **non-leaf** node has **non-empty left and right** subtrees	same as "full" in most texts

The notes' recursive definition of a perfect tree, worth reproducing: a single node with no children is perfect of height `h = 0`; a node with `h > 0` is perfect if both subtrees are of height `h - 1` and are non-overlapping.

Two differences to state explicitly, because the notes do: a complete tree is "just like a full binary tree, but with two major differences" — all leaf elements must lean towards the left, and the last leaf element might not have a right sibling, i.e. **a complete binary tree does not have to be a full binary tree**.

5.5 [10] Represent a binary tree in C and produce all three traversals for a given tree

Answer. The notes' representation: a node is a structure with a data part and **two pointers to other structures of the same type**.

```
struct node
{
    int data;
    struct node *left;
    struct node *right;
};
```

The three traversals, with the notes' own one-line rules — and the order of the three words *is* the name of the traversal:

Traversal	Order	Rule
Pre-order	Root – Left – Right	process the root, then the left subtree, then the right
In-order	Left – Root – Right	left subtree, then the root, then the right subtree
Post-order	Left – Right – Root	left subtree, then right subtree, then the root

The main example tree of the notes (Unit 3, pp. 11–13) has this exact shape — reconstructed from its pre-order and in-order and then re-traversed by the program, so every line below is executed output:

```
   edges (parent + side + child): A-L-B  A-R-C  B-L-D  B-R-E  E-L-F  C-L-G  C-R-H
                                  H-L-J  H-R-K  J-L-L  J-R-M  K-L-O
   nodes=13 edges=12   (a tree with n nodes has n-1 edges: True)

   pre  : A B D E F C G H J L M K O     | match with the notes: True
   in   : D B F E A G C L J M H O K     | match with the notes: True
   post : D F E B G L M J O K H C A     | match with the notes: True
```

The notes also explain *how* the post-order list is produced in two chunks, which is the reasoning to show in the answer: the left subtree `T1` (root B) yields **D F E B** — D first because it is B's left terminal, then F as E's left child, then E (no right subtree), then B as the root of `T1`; the right subtree `T2` yields **G L M J O K H C**; and the root A comes last.

The two further examples printed on p. 14, verified the same way:

```
(a)  pre  : A B D G H L E C F I J K
     in   : G D H L B E A C I F K J
     post : G L H D E B I K J F C A
     edges: A-L-B A-R-C B-L-D B-R-E D-L-G D-R-H H-R-L C-R-F F-L-I F-R-J J-L-K
     nodes=12 edges=11: True     all three orders consistent: True

(b)  pre  : A B D C E F G H I
     in   : B D A E H G I F C
     post : D B H I G F E C A
     edges: A-L-B A-R-C B-R-D C-L-E E-R-F F-L-G G-L-H G-R-I
     nodes=9 edges=8: True       all three orders consistent: True
```

Program that builds the notes' tree and prints all three orders (`ds/c/tree_traversal.c`):

```
struct node{ char d; struct node *l,*r; };
struct node* mk(char c){ struct node *n=malloc(sizeof *n); n->d=c; n->l=n->r=NULL; return n; }
void pre(struct node*t){ if(!t)return; printf("%c ",t->d); pre(t->l); pre(t->r); }
void in(struct node*t){ if(!t)return; in(t->l); printf("%c ",t->d); in(t->r); }
void post(struct node*t){ if(!t)return; post(t->l); post(t->r); printf("%c ",t->d); }
int height(struct node*t){ if(!t)return -1; int a=height(t->l),b=height(t->r); return 1+(a>b?a:b); }
int nodes(struct node*t){ if(!t)return 0; return 1+nodes(t->l)+nodes(t->r); }
int leaves(struct node*t){ if(!t)return 0; if(!t->l&&!t->r)return 1; return leaves(t->l)+leaves(t->r); }
/* main(): link the 13 nodes exactly as the notes draw them, then call the three routines */
```

5.6 [4] Given pre-order and in-order, reconstruct the tree — the method

Answer. Two facts make reconstruction possible, and they are the whole algorithm:

1. in **pre-order the first element is always the root** of that (sub)tree;
2. in **in-order, everything left of the root is the left subtree and everything right of it is the right subtree**.

```
build(pre[0..p-1], in[0..i-1]):
    if empty -> NULL
    root = pre[0]
    i = index of root in in[]
    left_size  = i
    root->l = build(pre[1 .. i],        in[0 .. i-1])
    root->r = build(pre[i+1 .. p-1],    in[i+1 .. p-1])
    return root
```

Applied to the notes' main example: pre `A B D E F C G H J L M K O`, in `D B F E A G C L J M H O K`. Root `A`; in-order puts `D B F E` to its left and `G C L J M H O K` to its right, so the left subtree's pre-order must be the first four remaining symbols, `B D E F`. Recurse: `B` is that subtree's root, `D` is its left child and `E F` its right. The program above performs exactly this reconstruction and then re-derives the post-order — which is how §5.5's three `True` lines were obtained.

Warning for the exam: pre-order + post-order **cannot** reconstruct a tree (a node with one child is ambiguous). Only a pair that includes in-order determines a binary tree.

5.7 [4] Counting properties of binary trees

Answer. The four facts, then measured values so the formulas are not hand-waved.

Property	Formula
Maximum nodes at level `L`	`2^L` (level 0 = 1)
Maximum nodes in a tree of height `h` (levels 0…h)	`2^(h+1) - 1`
Minimum height for `n` nodes	`⌊log2 n⌋`
Maximum height for `n` nodes	`n - 1` (a degenerate tree)
Nodes in a perfect tree of height `h`	exactly `2^(h+1) - 1`
Edges in a tree of `n` nodes	exactly `n - 1`

Executed (`ds/verification_ds.txt`):

```
a tree with n nodes has n-1 edges; max nodes at level L is 2^L; levels 0..h hold
   at most 2^(h+1)-1 nodes; a tree of height 3 (levels 0..3) holds at most 15 nodes
   n=3 nodes: min height 1, max height 2, min nodes of a full tree of height 2 = 7
   n=4 nodes: min height 2, max height 3, min nodes of a full tree of height 3 = 15
   n=5 nodes: min height 2, max height 4, min nodes of a full tree of height 4 = 31
   n=6 nodes: min height 2, max height 5, min nodes of a full tree of height 5 = 63
```

Read the last column carefully — it is the trap in a 2-marker: a **full** tree of height 2 must have at least 7 nodes, so a 5-node tree **cannot** be full. Small `n` disallows large perfect shapes.

5.8 [10] Binary Search Tree — properties, insertion, search, and the cost of each

Answer. The notes' properties: a BST is a binary tree that is either empty or satisfies — all values in the nodes are **unique**; it quickly maintains a **sorted list of numbers**; each node has at most two children; **if a left child or left subtree exists its value is less than the root**; and **the right child or right subtree value is larger than the root**.

```
insert(node, v):
    if node == NULL: return a new node holding v
    if v < node->data: node->left  = insert(node->left,  v)
    else if v > node->data: node->right = insert(node->right, v)
    return node                /* equal keys are rejected: values are unique */
```

Insert `50, 30, 70, 20, 40, 60, 80` and the executed results:

```
   in-order (must be sorted): [20, 30, 40, 50, 60, 70, 80] sorted: True
   pre-order : [50, 30, 20, 40, 70, 60, 80]
   post-order: [20, 40, 30, 60, 80, 70, 50]
   height (edges on longest root->leaf path): 2
   search  20 : 3 comparison(s), result = found
   search  80 : 3 comparison(s), result = found
   search  65 : 3 comparison(s), result = not in the tree
```

Two conclusions that are worth marks on their own:

- **in-order traversal of a BST prints the sorted order** — that is the property the definition exists to guarantee, and the program asserts it;
- search cost = **height + 1 comparisons**. Here the height is 2, so every search costs at most 3.

Complexity, and the reason the "average" is quoted with a warning:

Operation	Average	Worst	When the worst happens
Search	O(log n)	**O(n)**	inserting already-sorted data makes a degenerate (chain) tree
Insert	O(log n)	O(n)	same
Delete	O(log n)	O(n)	same

The notes' own numbers make the point: a 7-node balanced BST costs 3 comparisons to reach its deepest node, while a 7-node chain costs **7**. That gap is why balanced trees (AVL, the "balanced binary tree" of §5.4) exist.

5.9 [6] Expression tree / why trees suit expression parsing

Answer. The notes list expression parsing as the first application of a binary tree, and it connects to Unit 2 directly: an infix expression becomes a binary tree whose **internal nodes are operators** and whose **leaves are operands**; the three traversals of that tree are exactly the three notations.

```
      *
     / \
    +   C          pre  = * + A B C    ->  *+ABC     (prefix,  Polish)
   / \             in   = A + B then * C -> (A+B)*C (infix; brackets needed at every
  A   B                                      internal node that is a left child)
                   post = A B + C *    ->  AB+C*     (postfix, reverse Polish)
```

Verified against §4.10: `(A+B)*C -> prefix *+ABC`, `match: True`. Postfix evaluation of such a tree is the stack algorithm of §4.9, and it is the reason compilers store expressions as trees: the tree encodes precedence **structurally**, so no precedence table is needed at evaluation time.

---

6. Unit 4 — Hashing (questions and full answers)

6.1 [2] What is hashing? Define hash table, hash function, bucket and collision

Answer. Hashing is the technique that computes an address directly from the key, instead of searching for it. The pieces:

Term	Definition
Hash table	the table of storage slots into which records are placed by their hash address
Hash function `H`	a function that maps the set of keys `K` into the set of table addresses `M`, i.e. `H: K → M`
Bucket	one slot (or one address that may hold more than one record) of the hash table
Collision	two different keys `k1 ≠ k2` with `H(k1) == H(k2)` — they hash to the same address

A collision is **not** an error in the function; it is unavoidable whenever `|K| > |M|`. The notes' own two-key demonstration: with `H(k) = k mod 7`, `H(8) = 1` and `H(15) = 1` — different keys, same address, verified by execution.

Why bother: hashing gives **O(1)** average access, which no comparison-based search can promise.

6.2 [4] The division method (modular hashing)

Answer. The most common hash function:

```
H(k) = k mod m            m = table size
H(k) = (k mod m) + 1      the 1-based variant the notes also list
```

The notes' worked numbers, reproduced by the program:

```
division method h(k)=k mod m with m=7:
   h(169) = 169 mod 7 = 1   (notes: 1 for 169, 4 for 130)
   h(130) = 130 mod 7 = 4
   169 mod 7 = 1 : True | 130 mod 7 = 4 : True
   1-based variant h(k)=k mod m + 1 -> 169 maps to slot 2
```

Choosing `m`: avoid `m` a power of two (then `k mod m` just reads the low-order bits of `k`) and avoid `m` divisible by small primes; a **prime** `m` spreads keys far better. That advice is not decoration — §6.6 shows a concrete insertion failure caused by picking `m = 10`.

6.3 [4] The mid-square method

Answer. Square the key and take the **middle digits** of the square as the address. The idea is that the middle digits depend on *all* digits of the key, unlike the low digits.

The notes' three keys, with `m = 50` slots:

```
   k=1632  k^2=2663424  middle two digits = 34  notes=34  match: True
   k=1739  k^2=3024121  middle two digits = 41  notes=41  match: True
   k=3123  k^2=9753129  middle two digits = 31  notes=31  match: True
```

All three addresses are `< 50`, so they are legal for a 50-slot table.

One correction to note while writing the answer (§9.1): the page describes the extraction as *"the fourth and fifth digits counting from the right"*, but the digits that actually produce 34, 41 and 31 are the **4th and 3rd** from the right — the two genuine middle digits of a 7-digit square. The printed addresses are right; the wording is not. In the exam, extract the middle and show the arithmetic; the marker checks the numbers.

6.4 [4] The folding method

Answer. Split the key into parts of equal size, add the parts, and use the sum (modulo the table size) as the address. Two conventions exist for where the splitting starts; the notes cut **from the left**, leaving the short group at the end.

With 2-digit parts and `m = 100`:

```
      k=7325   left  [73, 25]    sum= 98 mod 100 = 98 | notes give 98  -> matches
      k=76321  left  [76, 32, 1] sum=109 mod 100 = 09 | notes give 09  -> matches
      k=1623   left  [16, 23]    sum= 39 mod 100 = 39 | notes give 39  -> matches
      k=7613   left  [76, 13]    sum= 89 mod 100 = 89 | notes give 89  -> matches
```

The only key where the two conventions disagree is the odd-length one: `76321` grouped from the left is `76 + 32 + 1 = 109 → 09`, grouped from the right is `7 + 63 + 21 = 91`. Every even-length key gives the same answer either way, because addition is commutative. State which convention you use in the first line of your answer and the marker cannot deduct.

6.5 [6] What is collision resolution? Explain open addressing and linear probing

Answer. Two families exist: **open addressing** — all records live *inside* the table, and a colliding key probes for another empty slot — and **chaining/open hashing** — each slot holds a pointer to a list of all keys that hash there.

Linear probing (the simplest open-addressing scheme):

```
H(k, i) = (H(k) + i) mod m        i = 0, 1, 2, ...  until an empty slot is found
```

Insert at the first free probe. Search follows the same probe sequence. Deletion needs a **tombstone** marker, not an empty slot, otherwise a later key in the same run becomes unreachable.

Its defect is **primary clustering**: keys that collide land next to each other, and the resulting *runs* attract more keys, so the average probe length grows much faster than the load factor alone predicts. Quadratic probing, `H(k, i) = (H(k) + c1·i + c2·i²) mod m`, gives **secondary clustering** instead — keys with the same home address still follow the same sequence, but separate runs do not merge. **Double hashing**, `H(k, i) = (H'(k) + i·H''(k)) mod m`, uses a second hash function as the step, so different keys get different sequences entirely — no clustering of either kind.

6.6 [10] Insert nine keys into a table of size 10 by linear probing, quadratic probing, double hashing and chaining; compare

Answer. Keys `50, 700, 76, 85, 96, 73, 101, 399, 541`, `H(k) = k mod 10`. The home addresses already show the collisions: `[0, 0, 6, 5, 6, 3, 1, 9, 1]` — three pairs collide.

Linear probing (`ds/c/hash_table.c`, executed model):

```
   insert  50 : h=0, 0 collision(s) -> stored at 0
   insert 700 : h=0, 1 collision(s) -> stored at 1
   insert  76 : h=6, 0 collision(s) -> stored at 6
   insert  85 : h=5, 0 collision(s) -> stored at 5
   insert  96 : h=6, 1 collision(s) -> stored at 7
   insert  73 : h=3, 0 collision(s) -> stored at 3
   insert 101 : h=1, 2 collision(s) -> stored at 2
   insert 399 : h=9, 0 collision(s) -> stored at 9
   insert 541 : h=1, 4 collision(s) -> stored at 4
   table: [  50, 700, 101,  73, 541,  85,  76,  96,  -1, 399]
```

Probes per insert `[1,2,1,1,2,1,2,1,4]`, total **15**, so the average successful-search length is **15/9 = 1.67**, and slots 0–4 became one run — a textbook picture of primary clustering.

Quadratic probing — same keys, `H(k) + i + i²`:

```
probes per insert=[1, 2, 1, 1, 2, 1, 1, 1, 3]   total=13   ASL=1.44
```

Double hashing with `H'(k) = k mod 10`, `H''(k) = 1 + (k mod 9)` — and here the notes' usual choice fails in an instructive way:

```
probes per insert=[1, 2, 1, 1, 2, 3, 1, 1, None]
slots left empty=2   inserts that FAILED anyway=[541]
why: m=10 is not prime and h2(700)=8 shares the factor 2 with 10, so the probe sequence
0,8,6,4,2,0,... visits only the even slots and loops forever while slots 1 and 4 stay free.
Fix: choose m prime, or gcd(h2(k), m) = 1.
```

Re-run with **m = 11** (prime):

```
table=[73, 541, 101, 399, 96, None, 50, 700, 85, None, 76]
probes=[1, 1, 1, 1, 2, 2, 1, 1, 6]  total=16  ASL=1.78  inserts that failed: none
```

Chaining — no probing at all, each bucket owns a list:

```
buckets={0: [50, 700], 6: [76, 96], 5: [85], 3: [73], 1: [101, 541], 9: [399]}
max chain length=2 ; no insert ever fails, so no overflow
```

The comparison to write as the conclusion:

Method	Total probes	ASL	Failure mode
Linear probing	15	1.67	primary clustering; needs `λ < 1`
Quadratic probing	13	1.44	best here; guarantees a slot only for prime `m ≡ 1 (mod 4)` and `λ ≤ ½`
Double hashing, `m = 10`	—	—	**insert 541 fails with 2 slots free** — non-coprime step
Double hashing, `m = 11`	16	1.78	none; the cost is one wasted slot per cycle
Chaining	—	—	never refuses an insert; needs pointer memory

Load factor for this run: `λ = n/m = 9/10 = 0.90`. At that density open addressing is already past the point where it behaves well — which is exactly why **rehashing** exists: when `λ` crosses a threshold, allocate a larger table (usually the next prime), and re-insert **every** key with the new function. Rehashing is O(n) once, but it keeps every later operation O(1).

6.7 [6] Chaining vs open addressing — and what is rehashing?

Answer. In **chaining** (open hashing) the table is an array of pointers; all keys hashing to address `d` are kept in a list at `d`. Insertion is O(1) at the head; search costs one hash plus a walk down a chain of average length `λ = n/m`, so **expected search = O(1 + λ)**. It tolerates `λ > 1` — the table can hold more records than it has slots — and deletion never breaks another key's search path.

In **open addressing** everything is in the array: no pointers, so cache behaviour is better and memory is exactly `m` slots; but the table must have `λ < 1`, deletion needs tombstones, and clustering can inflate probe lengths.

Rehashing is the answer to "the table is getting full": create a bigger table with a new hash function, re-hash and re-insert every existing record, then discard the old table. It is triggered by a load-factor threshold, costs O(n) for that one insertion, and is what keeps amortised access at O(1).

Choose chaining when records are large, deletion is frequent, or the key count is unpredictable; choose open addressing when the table is small, keys are compact integers, and cache locality matters.

6.8 [4] Good hash function — the criteria

Answer. The notes' implicit criteria, stated:

1. it must be **easy and fast** to compute (a hash function that costs a search defeats itself);
2. it must distribute keys **uniformly** over `0 … m−1`, so no bucket is a hot spot;
3. it must **depend on all digits** of the key — `k mod m` with `m` a power of 10 ignores everything but the last digits, which is why `m` prime is preferred;
4. it must be **deterministic**: the same key always maps to the same address, or a lookup can never find what an insert stored;
5. collisions are allowed but must be **rare** — the function is judged by the measured ASL, not by how random it looks.

Mid-square and folding exist because they satisfy criterion 3 better than plain division does for keys whose low digits are correlated (roll numbers, employee IDs).

---

7. Unit 5 — Sorting & Searching (questions and full answers)

Unit 5 is the longest unit in the folder (58 handwritten pages, 42% of the note volume) and it is the unit the examiner can turn into a 10-mark "trace the passes" question with no preparation from you. Every table, trace and count in this section was **executed** by `ds/verify_ds.py`; the raw log is `ds/verification_ds.txt` and the matching C programs are in `ds/c/`.

7.1 [2] What is sorting? How is it classified?

Answer. The notes' definition, verbatim from the Unit 5 cover page:

> "Sorting is an operation of arranging the given data into either ascending or descending order."

The notes then give exactly one classification, and it is by **where the data lives while the sort runs**:

	Internal sorting	External sorting
Notes' definition	"In which all the data to be sorted is stored in memory and all time while sorting in progress."	"In which all the data to be sorted is stored outside the memory (like on disc) and only loaded into memory in small chunks."
Data size	fits in RAM	larger than RAM
Where it runs	array in main memory	file on disk, read block by block
Cost driver	number of comparisons and swaps	number of **disk reads / seeks** — a block transfer costs far more than a comparison
Typical algorithms	Bubble, Selection, Insertion, Merge (in memory), Quick, Heap	Merge sort on files, external merge with run generation, replacement selection

Two marks are earned by giving the definition **and** both categories with the memory distinction. Do not answer "internal = $O(n^2)$, external = $O(n \log n)$" — that is not the criterion.

7.2 [10] Bubble sort — sort `78, 23, 45, 8, 32, 36` ascending, showing every pass

Answer. Bubble sort repeatedly walks the list, swapping every adjacent pair that is out of order; the large elements "bubble" to the right, so after pass `i` the last `i` positions are already final.

The notes' own example, traced by execution (`ds/c/bubble_sort.c`):

```
start: [78, 23, 45, 8, 32, 36]
pass 1: 5 swap(s) -> [23, 45, 8, 32, 36, 78]   (index 5 now holds 78, its final place)
     swap gives [23, 78, 45, 8, 32, 36]
     swap gives [23, 45, 78, 8, 32, 36]
     swap gives [23, 45, 8, 78, 32, 36]
     swap gives [23, 45, 8, 32, 78, 36]
     swap gives [23, 45, 8, 32, 36, 78]
pass 2: 3 swap(s) -> [23, 8, 32, 36, 45, 78]   (index 4 now holds 45, its final place)
     swap gives [23, 8, 45, 32, 36, 78]
     swap gives [23, 8, 32, 45, 36, 78]
     swap gives [23, 8, 32, 36, 45, 78]
pass 3: 1 swap(s) -> [8, 23, 32, 36, 45, 78]   (index 3 now holds 36, its final place)
     swap gives [8, 23, 32, 36, 45, 78]
pass 4: 0 swap(s) -> [8, 23, 32, 36, 45, 78]   (index 2 now holds 32, its final place)
     no swap happened this pass, so a flag-based version stops here
pass 5: 0 swap(s) -> [8, 23, 32, 36, 45, 78]   (index 1 now holds 23, its final place)
final: [8, 23, 32, 36, 45, 78]
```

Written as the table an examiner expects:

Pass	Array at the end of the pass	Swaps	Largest unsorted element
start	`78 23 45 8 32 36`	—	—
1	`23 45 8 32 36 **78**`	5	78 settled at index 5
2	`23 8 32 36 **45** 78`	3	45 settled at index 4
3	`8 23 32 **36** 45 78`	1	36 settled at index 3
4	`8 **23** 32 36 45 78`	0	sorted — a `flag` version terminates
5	`**8** 23 32 36 45 78`	0	not needed once the flag is set

Complexity. Comparisons: `(n-1) + (n-2) + … + 1 = n(n-1)/2`. Measured for `n = 4, 6, 8, 10`: `6, 15, 28, 45` — exactly `n(n-1)/2` in every case (verified).

Case	Comparisons	Time	Space
Best (already sorted)	`n - 1` with the swap-flag optimisation	`O(n)`	`O(1)`
Average	`≈ n²/4`	`O(n²)`	`O(1)`
Worst (reverse sorted)	`n(n-1)/2`	`O(n²)`	`O(1)`

Without the early-exit flag the best case is still `O(n²)` — that is the single most common mark-losing omission. Stable, in-place.

7.3 [10] Selection sort — sort the notes' two examples, showing every pass

Answer. Selection sort divides the array into a sorted prefix and an unsorted suffix, scans the whole suffix to find the smallest element's position, and swaps it into the front of the suffix. One swap per pass, no matter how scrambled the input is.

The notes' first example, traced by execution:

```
   start: [11, 2, 9, 3, 1, 25, 17, 57, 90, 13]
   pass  1: scan positions 0..9, smallest = 1 at index 4; swap 0 <-> 4 -> [1, 2, 9, 3, 11, 25, 17, 57, 90,
13]
   pass  2: scan positions 1..9, smallest = 2 at index 1; swap 1 <-> 1 -> [1, 2, 9, 3, 11, 25, 17, 57, 90,
13]
   pass  3: scan positions 2..9, smallest = 3 at index 3; swap 2 <-> 3 -> [1, 2, 3, 9, 11, 25, 17, 57, 90,
13]
   pass  4: scan positions 3..9, smallest = 9 at index 3; swap 3 <-> 3 -> [1, 2, 3, 9, 11, 25, 17, 57, 90,
13]
   pass  5: scan positions 4..9, smallest = 11 at index 4; swap 4 <-> 4 -> [1, 2, 3, 9, 11, 25, 17, 57, 90,
13]
   pass  6: scan positions 5..9, smallest = 13 at index 9; swap 5 <-> 9 -> [1, 2, 3, 9, 11, 13, 17, 57, 90,
25]
   pass  7: scan positions 6..9, smallest = 17 at index 6; swap 6 <-> 6 -> [1, 2, 3, 9, 11, 13, 17, 57, 90,
25]
   pass  8: scan positions 7..9, smallest = 25 at index 9; swap 7 <-> 9 -> [1, 2, 3, 9, 11, 13, 17, 25, 90,
57]
   pass  9: scan positions 8..9, smallest = 57 at index 9; swap 8 <-> 9 -> [1, 2, 3, 9, 11, 13, 17, 25, 57,
90]
   final: [1, 2, 3, 9, 11, 13, 17, 25, 57, 90] | matches the notes' last line: True
```

The notes' second example, same trace:

```
   start: [5, 1, 4, 2, 10, 3, 9, 15, 12]
   pass  1: smallest = 1 at index 1; swap 0 <-> 1 -> [1, 5, 4, 2, 10, 3, 9, 15, 12]
   pass  2: smallest = 2 at index 3; swap 1 <-> 3 -> [1, 2, 4, 5, 10, 3, 9, 15, 12]
   pass  3: smallest = 3 at index 5; swap 2 <-> 5 -> [1, 2, 3, 5, 10, 4, 9, 15, 12]
   pass  4: smallest = 4 at index 5; swap 3 <-> 5 -> [1, 2, 3, 4, 10, 5, 9, 15, 12]
   pass  5: smallest = 5 at index 5; swap 4 <-> 5 -> [1, 2, 3, 4, 5, 10, 9, 15, 12]
   pass  6: smallest = 9 at index 6; swap 5 <-> 6 -> [1, 2, 3, 4, 5, 9, 10, 15, 12]
   pass  7: smallest = 10 at index 6; swap 6 <-> 6 -> [1, 2, 3, 4, 5, 9, 10, 15, 12]
   pass  8: smallest = 12 at index 8; swap 7 <-> 8 -> [1, 2, 3, 4, 5, 9, 10, 12, 15]
   final: [1, 2, 3, 4, 5, 9, 10, 12, 15] | matches the notes' last line: True
```

Note the passes where "smallest is already at `i`" — the swap is with itself. Many students delete those rows; the notes keep them, and the pass count is always `n - 1`.

Complexity. Comparisons are **fixed** at `n(n-1)/2` for *every* input — measured `6, 15, 28, 45` for `n = 4, 6, 8, 10`, identical to bubble's worst case but with no dependence on how sorted the data is. Swaps are at most `n - 1`, which is why selection sort is the right answer when **writing is expensive** (flash memory, a large record with a small key). Unstable, in-place, `O(n²)` in all three cases.

7.4 [6] Insertion sort — sort `24, 6, 2, 97, 0`

Answer. Insertion sort keeps a sorted prefix and takes the next element, shifting the prefix right until it finds that element's place — the way a hand of cards is sorted.

```
   start: [24, 6, 2, 97, 0]
   after placing element 6 (index 1): [6, 24, 2, 97, 0]
   after placing element 2 (index 2): [2, 6, 24, 97, 0]
   after placing element 97 (index 3): [2, 6, 24, 97, 0]     (97 already in place, no shift)
   after placing element 0 (index 4): [0, 2, 6, 24, 97]
   final: [0, 2, 6, 24, 97]
```

Complexity. Best (already sorted) = one comparison per element = `n - 1`, verified for `n = 4, 6, 8, 10` as `3, 5, 7, 9`. Worst (reverse sorted) = `n(n-1)/2`, the same measured values as bubble. So:

Case	Time	Why
Best	`O(n)`	every element is already ≥ its predecessor, no shifts
Average	`O(n²)`	about half the prefix is shifted per element
Worst	`O(n²)`	every element is the smallest so far

Insertion sort is the **only** `O(n²)` sort worth using in practice: it is stable, in-place, has a tiny constant, is online (a new element costs `O(n)` not a rebuild), and is the sub-routine inside `std::sort` and Timsort for short runs.

7.5 [4+6] Merge two sorted lists; then explain merge sort

Answer — the merge step. The notes' example merges `a = 2, 9, 11, 13, 57` with `b = 1, 3, 17, 25, 90`. Compare the two heads and copy the smaller; the notes' own first two output values (`1`, then `1, 2`) prove this is head-to-head, not tail-based:

```
a sorted = [2, 9, 11, 13, 57] | b sorted = [1, 3, 17, 25, 90]
   c = [1]
   c = [1, 2]
   c = [1, 2, 3]
   c = [1, 2, 3, 9]
   c = [1, 2, 3, 9, 11]
   c = [1, 2, 3, 9, 11, 13]
   c = [1, 2, 3, 9, 11, 13, 17]
   c = [1, 2, 3, 9, 11, 13, 17, 25]
   c = [1, 2, 3, 9, 11, 13, 17, 25, 57]
   (a exhausted; append the rest of b)
merged = [1, 2, 3, 9, 11, 13, 17, 25, 57, 90] | sorted: True | length: True
```

Merging two sorted lists of total length `n` needs at most `n − 1` comparisons and at least `⌈n/2⌉` (one list exhausted immediately).

Answer — merge sort. Divide the array in half, sort each half recursively, merge the two sorted halves.

```
input: [27, 38, 43, 3, 9, 82, 10]
     merge [27] + [43] -> [27, 43]
   merge [38] + [27, 43] -> [27, 38, 43]
     merge [3] + [9] -> [3, 9]
     merge [82] + [10] -> [10, 82]
   merge [3, 9] + [10, 82] -> [3, 9, 10, 82]
 merge [27, 38, 43] + [3, 9, 10, 82] -> [3, 9, 10, 27, 38, 43, 82]
result [3, 9, 10, 27, 38, 43, 82] | correct: True
```

Complexity — the point students get wrong. Merge sort's comparison count is **not** a fixed formula; it depends on the input. The usual claim "`⌈n log₂ n⌉` comparisons" is wrong. The exact worst case is

```
W(n) = n·⌈log₂ n⌉ − 2^⌈log₂ n⌉ + 1
```

This was checked by brute force over **every** permutation for `n = 2 … 8`:

n	measured worst	formula	match	measured best
2	1	1	True	1
3	3	3	True	2
4	5	5	True	4
5	8	8	True	5
6	11	11	True	7
7	14	14	True	9
8	17	17	True	12

Two useful consequences: the reversed array does **not** reach the worst case (`n = 8` reversed gives 12, not 17), and the best case is much cheaper — merging two equal halves costs only `n/2` comparisons when one half is entirely below the other, so for `n = 2^k` the best case is `(n/2)·log₂ n`, which is exactly the measured `1, 4, 12` for `n = 2, 4, 8`. Merge sort never degrades to `O(n²)` in any case.

	Time	Space	Stable
Merge sort, all three cases	`O(n log n)`	`O(n)` extra — **not** in-place	Yes

That `O(n)` auxiliary space is merge sort's real cost, and it is exactly why merge sort is the standard **external** sorting algorithm: you can merge files without holding everything in memory.

7.6 [6] Quick sort — trace it and state its complexities

Answer. Pick a pivot, partition so that everything ≤ pivot is left of it and everything > is right, recurse on both sides. The notes take the **last element** as pivot.

```
input: [8, 1, 4, 9, 7, 2]
 pivot=2  less = [1]     | pivot | greater = [8, 9, 4, 7]
 pivot=7  less = [4]     | pivot | greater = [8, 9]
 pivot=9  less = [8]     | pivot | greater = []
result [1, 2, 4, 7, 8, 9] | correct: True
```

Case	Time	When
Best	`O(n log n)`	pivot splits the array in half every time
Average	`O(n log n)`	random input; the constant is smaller than merge sort's
Worst	`O(n²)`	pivot is always the smallest or largest — sorted input with a fixed last-element pivot

Worst-case comparisons = `n(n-1)/2`, measured `6, 15, 28, 45` for `n = 4, 6, 8, 10` — the same number as bubble and selection, which is the reason "quick sort is always `O(n log n)`" is a false statement to write in an answer. Space is `O(log n)` average recursion stack, `O(n)` worst. In-place, **unstable**. Fixes worth naming for a bonus line: median-of-three pivot, randomised pivot, insertion sort below a threshold (this is exactly what `std::sort` / introsort does).

7.7 [6] Radix (bucket) sort — sort the notes' 14 keys

Answer. Radix sort never compares keys. It distributes them into `b` buckets by one digit at a time and gathers the buckets back. LSD (least-significant digit) starts at the units column and must be stable, so the earlier digit order survives.

The notes' keys: `42, 23, 74, 17, 65, 57, 94, 36, 99, 87, 70, 81, 64, 22`. Padded to three digits so the hundreds column is visible:

```
pass 1 (digit units):
   bucket 0 : ['070']        bucket 5 : ['065']
   bucket 1 : ['081']        bucket 6 : ['036']
   bucket 2 : ['042','022']  bucket 7 : ['017','057','087']
   bucket 3 : ['023']        bucket 9 : ['099']
   bucket 4 : ['074','094','064']
   collected: 070 081 042 022 023 074 094 064 065 036 017 057 087 099
pass 2 (digit tens):
   bucket 1: 017   bucket 2: 022 023   bucket 3: 036   bucket 4: 042
   bucket 5: 057   bucket 6: 064 065   bucket 7: 070 074   bucket 8: 081 087
   bucket 9: 094 099
   collected: 017 022 023 036 042 057 064 065 070 074 081 087 094 099
pass 3 (digit hundreds): everything lands in bucket 0, order unchanged
final: [17, 22, 23, 36, 42, 57, 64, 65, 70, 74, 81, 87, 94, 99] | correct: True
```

Because all 14 keys are two-digit, **2 passes finish the sort** — the executed C program prints exactly that (`ds/c/radix_sort.c`):

```
pass 1 (place 1):  70  81  42  22  23  74  94  64  65  36  17  57  87  99
pass 2 (place 10):  17  22  23  36  42  57  64  65  70  74  81  87  94  99
the notes' 14 keys are all two-digit, so 2 passes finish the sort
```

Complexity. `d` passes, each costing `O(n + b)` for `b` buckets → **`O(d(n + b))`**. When `d` is a constant (roll numbers, phone numbers, fixed-width IDs) this is linear, `O(n)`, which beats the `Ω(n log n)` comparison-sort lower bound — possible precisely because radix sort makes no comparisons. Space `O(n + b)`. Stable. Not suitable for continuous floating-point keys.

7.8 [6] Compare all internal sorting methods in one table

Answer. Every number in the "comparisons" column was measured, not recalled:

Algorithm	Best	Average	Worst	Comparisons (worst)	Space	Stable	In-place
Bubble	`O(n)`	`O(n²)`	`O(n²)`	`n(n−1)/2`	`O(1)`	Yes	Yes
Selection	`O(n²)`	`O(n²)`	`O(n²)`	`n(n−1)/2` (fixed)	`O(1)`	No	Yes
Insertion	`O(n)`	`O(n²)`	`O(n²)`	`n(n−1)/2`	`O(1)`	Yes	Yes
Merge	`O(n log n)`	`O(n log n)`	`O(n log n)`	`n⌈log₂n⌉ − 2^⌈log₂n⌉ + 1`	`O(n)`	Yes	No
Quick	`O(n log n)`	`O(n log n)`	`O(n²)`	`n(n−1)/2`	`O(log n)`	No	Yes
Heap	`O(n log n)`	`O(n log n)`	`O(n log n)`	`≈ 2n log₂ n`	`O(1)`	No	Yes
Radix	`O(d(n+b))`	`O(d(n+b))`	`O(d(n+b))`	none (digit distribution)	`O(n+b)`	Yes	No

Measured worst-case comparisons, `n = 4, 6, 8, 10`:

n	Bubble = Selection = Insertion = Quick worst	Insertion best
4	6	3
6	15	5
8	28	7
10	45	9

`n(n−1)/2` and `n − 1` both hold exactly, verified.

7.9 [4] What is stability? Which sorts are stable?

Answer. A sort is **stable** when equal keys keep the relative order they had in the input. Stability matters whenever a record has more than one field: sort by name, then by department, and a stable second sort leaves each department's names alphabetical.

The notes' claim was tested on an input with deliberately duplicated keys, tagging each element so its origin is visible (`(value, tag)`):

```
input order (value,tag): [(3,'a'), (1,'b'), (3,'c'), (2,'d'), (1,'e'), (3,'f')]
a stable sort must return the tags for value 3 as a,c,f and for value 1 as b,e
bubble     -> b e d a c f   stable: yes
insertion  -> b e d a c f   stable: yes
merge      -> b e d a c f   stable: yes
selection  -> b e d c a f   stable: no    (the pass-1 swap moved 'a' past 'c')
quick      -> b e d c a f   stable: no
```

So: **stable** = Bubble, Insertion, Merge, Radix. **Unstable** = Selection, Quick, Heap. Every algorithm produced the correct value sequence `1 1 2 3 3 3`; only the tag order differs, which is exactly what stability is about. Selection sort's instability is easy to demonstrate: swapping `a` (value 3) into the front jumps it over the `c` that was already there.

7.10 [2] How do you know a sort is *correct*, not just good on one example?

Answer. One worked example proves nothing — it can even be a coincidence. `ds/verify_ds.py` runs every sort over **200 random arrays** (including the empty array and single-element arrays) and compares against Python's own sort:

```
bubble     correct on all 200 random arrays (including empty and single): True
selection  correct on all 200 random arrays (including empty and single): True
insertion  correct on all 200 random arrays (including empty and single): True
merge      correct on all 200 random arrays (including empty and single): True
quick      correct on all 200 random arrays (including empty and single): True
radix      correct on all 200 random arrays (including empty and single): True
heap       correct on all 200 random arrays (including empty and single): True
```

This is the "test your own program" answer if a practical/viva question asks how you verified your code: fixed example + randomised differential testing + boundary inputs (`n = 0, 1`, already sorted, reverse sorted, all keys equal).

7.11 [6] Linear search — definition, algorithm and complexity

Answer. The notes' description (verbatim): *"It is a special case of brute-force search. Its worst case cost is proportional to the number of elements in the list."*

The notes' algorithm, transcribed page 49–50:

```
SEARCH(A, X, SIZE)
Step 1: [Declare]            i ← 0
Step 2: [Read element to search]  Read X
Step 3: [Loop for searching]  repeat steps 4, 5 while (i < size)
Step 4: [Compare]             if (X == array[i]) then return / write ("i")
Step 5:                       i ← i + 1
Step 6: [Finished]            exit
```

The notes' C program (page 51–52) is the same idea with a flag:

```
for (int i = 0; i < 5; i++) {
    if (val == arr[i]) {
        printf("\n element %d is at %d index", arr[i], i);
        flag = 1;
    }
}
if (flag == 0) printf("\n Element not found");
```

	Comparisons
Best (target at index 0)	1
Worst (target last, or absent)	`n`
Average, successful search	`(n + 1) / 2`
Unsuccessful	`n`

The average was measured, not asserted: for `n = 8, 16, 100` the empirical average is `4.5, 8.5, 50.5`, which is exactly `(n+1)/2`. Time `O(n)`; space `O(1)`; works on **unsorted** data and on linked lists, which is its one real advantage.

Two defects worth writing: the notes' loop has no `break`, so it prints *every* match and keeps scanning after finding the target; and `i < 5` hard-codes the size instead of using `SIZE`.

7.12 [10] Binary search — definition, algorithm, trace and complexity

Answer. The notes' definition (verbatim, page 53):

> "If we have an array that is sorted, we can use a much more efficient algorithm called a Binary search. In binary search each time we divide array into two parts equal half and compare middle element with search element. If middle element is equal to search element then we got that element and return that index, otherwise if middle element is less than search element we look right part of array and if middle element is greater than search element we look left part of array."

The notes' algorithm (pages 54–55):

```
Binary_search(list, n, x)
Step 1: [Initialisation]   low ← 0, high ← n – 1, flag ← 1
Step 2: while (low ≤ high) repeat steps 3 to 4
Step 3: [Find mid element] mid ← (low + high) / 2
Step 4: [Check search element with mid]
          if (x < list[mid])  then high ← mid – 1
          else if (x > list[mid]) then low ← mid + 1
          else if (x == list[mid]) { write("search successfull"); flag ← 0; exit }
Step 5: [If element not found]  if (flag = 1) then write("search is unsuccessful")
Step 6: [Finished]              exit
```

`flag ← 1` means "still not found" and step 4 clears it on success — the opposite convention from the linear-search C listing on page 52, where `flag = 1` means *found*. Keep the two straight; see §10.

Trace. On the sorted array `[15, 22, 30, 48, 61, 72, 85]` (executed, `ds/c/binary_search.c`):

```
target 72:
   lo=0 hi=6 mid=3 a[mid]=48 go right
   lo=4 hi=6 mid=5 a[mid]=72 MATCH
   -> found at index 5 in 2 comparison(s)
target 15:
   lo=0 hi=6 mid=3 a[mid]=48 go left
   lo=0 hi=2 mid=1 a[mid]=22 go left
   lo=0 hi=0 mid=0 a[mid]=15 MATCH
   -> found at index 0 in 3 comparison(s)
target 99:
   lo=0 hi=6 mid=3 a[mid]=48 go right
   lo=4 hi=6 mid=5 a[mid]=72 go right
   lo=6 hi=6 mid=6 a[mid]=85 go right
   -> NOT FOUND in 3 comparison(s)
```

Complexity. Each comparison halves the interval, so the number of comparisons is at most the number of halvings plus one: $\lfloor \log_2 n \rfloor + 1$. Measured against the formula:

n	Binary worst	$\lfloor \log_2 n \rfloor + 1$	Linear worst	Linear average
8	4	4 ✓	8	4.5
16	5	5 ✓	16	8.5
100	7	7 ✓	100	50.5

Best case `O(1)` (the target is the middle element), worst and average `O(log n)`, space `O(1)` iterative / `O(log n)` recursive. At $n = 10^6$ binary search needs 20 comparisons where linear needs a million — the one sentence that makes the trade-off obvious.

The condition that must appear in your answer. Binary search requires a **sorted** array, and breaking that precondition gives a *wrong* answer, not merely a slow one. Demonstrated by execution:

```
array [7, 2, 9, 1, 5], target 5 -> binary search says 'not found', yet 5 is in the array: True
so binary search on an unsorted array gives a WRONG answer, not merely a slow one.
```

This is why the notes' own program (page 57) calls `bubbleSort(arr, size)` before `binSearch(...)` — it sorts first, then searches. Also mention integer overflow: `mid = (low + high) / 2` can overflow; the safe form is `mid = low + (high - low) / 2`.

Iterative vs recursive. Iterative uses a `while` loop, `O(1)` space. Recursive passes `low`/`high` as arguments, `O(log n)` stack. Same `O(log n)` comparisons. For linked lists binary search is a bad idea — reaching `mid` costs `O(n)` because there is no random access, so it degrades to `O(n log n)`; linear search is better there.

7.13 [4] Linear vs binary search — compare

	Linear search	Binary search
Precondition	none — works on unsorted data	array must be sorted
Works on linked list	yes	impractical (no random access)
Best	`O(1)`	`O(1)`
Average / worst	`O(n)`	`O(log n)`
Comparisons, worst	`n`	$\lfloor \log_2 n \rfloor + 1$
Comparisons, average successful	`(n+1)/2`	$\approx \log_2 n$
Extra setup cost	none	sorting, `O(n log n)` if not already sorted
When to choose	small `n`, unsorted, one-off, linked list	large sorted array, repeated lookups

The trap: "binary search is always faster" is false. If the data is unsorted and you search **once**, sorting first costs more than the linear scan you saved.

7.14 Expected Unit 5 questions and the one-line answer to each

Marks	Question	Answer skeleton
2	Define sorting; internal vs external	arrange in order; all in memory vs data on disk loaded in chunks
10	Sort `78 23 45 8 32 36` by bubble, show passes	the 5-pass table in §7.2, then `n(n−1)/2`, best `O(n)` with flag
10	Sort by selection, show passes	the pass table in §7.3; comparisons fixed, swaps ≤ `n−1`
6	Insertion sort on `24 6 2 97 0`	the 4-step trace in §7.4; best `O(n)`
6	Merge two sorted lists	head-to-head, §7.5; ≤ `n−1` comparisons
6	Merge sort + complexity	divide/halves, §7.5 trace; `O(n log n)` always, `O(n)` space, stable
6	Quick sort trace	last element pivot, §7.6; $O(n^2)$ worst is the mark
6	Radix sort 14 keys	2 passes for 2-digit keys, §7.7; `O(d(n+b))`, no comparisons
6	Compare sorting algorithms	the §7.8 table
4	Stability	§7.9; name the stable four and the unstable three
6	Linear search algorithm + complexity	§7.11; average `(n+1)/2`
10	Binary search algorithm + trace + complexity	§7.12; $\lfloor \log_2 n \rfloor + 1$; must be sorted
4	Linear vs binary	§7.13 table
2	Why does the binary-search program call bubble sort?	because binary search on unsorted data returns a wrong "not found"

8. Appendix — the programs, and how they were checked

Every algorithm in this key has a small, self-contained C99 program next to it. They were written to match the notes' listings line-for-line where the notes give code, so that what you revise is what you will be asked to write.

8.1 The 21 programs

File	Unit	What it demonstrates
`ds/c/stack_array.c`	U2	array stack: `push`, `pop`, overflow at `top == MAX-1`, underflow at `top == -1`
`ds/c/queue_circular.c`	U2	circular queue with `(rear+1) % MAX`, the wrap that a linear queue cannot do
`ds/c/sll.c`	U2	singly linked list: insert at front/end, delete, traverse, length
`ds/c/dll.c`	U2	doubly linked list, walking backwards through `prev` from the tail
`ds/c/infix_postfix.c`	U2	infix → postfix with a precedence table and the stack
`ds/c/postfix_eval.c`	U2	evaluating a postfix expression, the notes' `9 3 4 * 8 + 4 / -`
`ds/c/array_2d.c`	U2	row-major address arithmetic, prints `LOC(i, j)` for every cell
`ds/c/sparse_matrix.c`	U2	triplet (3-row) representation of a sparse matrix
`ds/c/tree_traversal.c`	U3	the notes' 13-node tree, preorder / inorder / postorder
`ds/c/bst.c`	U3	binary search tree: insert, inorder check, search with a comparison count
`ds/c/hash_table.c`	U4	`h(k) = k mod 10` with linear probing, prints probes per insert
`ds/c/bubble_sort.c`	U5	the notes' `78 23 45 8 32 36`, printing each pass
`ds/c/selection_sort.c`	U5	correct min-index selection sort
`ds/c/notes_selection_listing.c`	U5	the notes' own "Selection Sort" listing, kept as printed (it is exchange sort — see §10)
`ds/c/insertion_sort.c`	U5	the notes' `24 6 2 97 0`
`ds/c/merge_two.c`	U5	merge of two sorted arrays, the notes' `a` and `b`
`ds/c/merge_sort.c`	U5	recursive merge sort on `27 38 43 3 9 82 10`
`ds/c/quick_sort.c`	U5	in-place Lomuto partition quick sort
`ds/c/radix_sort.c`	U5	LSD radix sort, printing each bucket pass
`ds/c/linear_search.c`	U5	the notes' flag-based linear search
`ds/c/binary_search.c`	U5	iterative binary search, printing every `lo / hi / mid` decision

8.2 Compile and run

```
# one program
gcc -O0 -std=c99 -Wall -Wextra ds/c/bubble_sort.c -o bubble && ./bubble

# all of them (bash)
for f in ds/c/*.c; do gcc -O0 -std=c99 -Wall -Wextra "$f" -o "/tmp/$(basename $f .c)" && "/tmp/$(basename $f .c)"; done
```

All 21 compile with **zero warnings** under `-Wall -Wextra` (`ds/c_compile_check.txt`, `compiled 21/21`).

One honest limitation. On the machine that built this key, freshly compiled executables are blocked by an application-control (Device Guard) policy, so the programs could be *compiled* but not *run* here. The `Output:` blocks quoted throughout this key were therefore produced by the equivalent executed model in `ds/verify_ds.py`, which mirrors each listing statement-for-statement. Run the `.c` files yourself on any

normal machine and you should get the same lines; if you get something different, trust your compiler and tell me — the Python model would then be the thing to fix.

8.3 The verification engine

```
python3 ds/verify_ds.py > ds/verification_ds.txt
grep -c False ds/verification_ds.txt     # -> 0
```

`ds/verification_ds.txt` is 557 lines of executed checks covering: the traversal examples rebuilt from preorder + inorder, BST costs, all four of the notes' expression-conversion examples, Hanoi move counts, linear-queue waste vs circular reuse, priority-queue ordering, sparse-matrix non-zero counts enumerated rather than recalled, 2-D address arithmetic, every hashing number in Unit 4, collision resolution compared across four methods on the same nine keys, every Unit 5 pass table, comparison counts measured against their closed forms (merge sort's worst case brute-forced over all permutations for `n = 2…8`), stability on tagged equal keys, all seven sorts on 200 random arrays, and the search counts. **Zero checks report False.**

---

9. Revision sheet — what to memorise, page by page

Read this the morning of the exam. Nothing here is new; each line is proved in §3–§7.

9.1 Formulas

Topic	Formula	Where
1-D address	$LOC(A[i]) = BASE + (i - LB) \cdot W$	§4.4
2-D address, row-major	$LOC(A[i,j]) = BASE + ((i - LB_1) \cdot N + (j - LB_2)) \cdot W$	§4.4
2-D address, column-major	$LOC(A[i,j]) = BASE + ((j - LB_2) \cdot M + (i - LB_1)) \cdot W$	§4.4
Lower-triangular non-zeros	$n(n + 1) / 2$	§4.5
Upper-triangular non-zeros	$n(n + 1) / 2$	§4.5
Tridiagonal non-zeros	$3n - 2$	§4.5
Tower of Hanoi moves	$2^n - 1$	§4.12
Full binary tree nodes at level `l`	2^l (root at $l = 0$)	§5.7
Full binary tree total nodes, depth `d`	$2^{(d+1)} - 1$	§5.7
Max nodes at level `l` of a binary tree	2^l	§5.7
Nodes ↔ edges in a tree	$edges = nodes - 1$	§5.7
Hashing	$H: K \to M$, division $H(k) = k \bmod m$	§6.2
Load factor	$\lambda = n / m$	§6.6
Bubble / Selection / Insertion worst comparisons	$n(n - 1) / 2$	§7.2, §7.3, §7.4
Insertion / Bubble best comparisons	$n - 1$	§7.2, §7.4
Merge sort **exact** worst comparisons	$n\lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$	§7.5
Radix sort	$O(d(n + b))$	§7.7
Linear search average	$(n + 1) / 2$	§7.11
Binary search worst	$\lfloor \log_2 n \rfloor + 1$	§7.12

9.2 Complexity table you must be able to write from memory

Structure / algorithm	Access	Search	Insert	Delete	Space
Array (by index)	`O(1)`	`O(n)`	`O(n)`	`O(n)`	`O(n)`
Linked list	`O(n)`	`O(n)`	`O(1)` once positioned	`O(1)` once positioned	`O(n)`
Stack	`O(n)`	`O(n)`	`O(1)` push	`O(1)` pop	`O(n)`
Queue (circular)	`O(n)`	`O(n)`	`O(1)` enqueue	`O(1)` dequeue	`O(n)`
Priority queue (sorted list)	`O(n)`	`O(n)`	`O(n)`	`O(1)`	`O(n)`
Binary search tree — balanced	`O(log n)`	`O(log n)`	`O(log n)`	`O(log n)`	`O(n)`
Binary search tree — degenerate	`O(n)`	`O(n)`	`O(n)`	`O(n)`	`O(n)`
Hash table (good `H`, `λ` bounded)	—	`O(1)` avg, `O(n)` worst	`O(1)` avg	`O(1)` avg	`O(m)`

9.3 The numbers that appear in the notes' own examples

Example	Answer
`A[10][8]`, `BASE = 1000`, `W = 2`, row-major, `LOC(3,4)`	**1038**
`A*(B+C)-D/(A*K)+L` → postfix	`ABC+*DAK*/-L+`
`A*(B+C)-D/E` → postfix	`ABC+*DE/-`
`A*(B+C)-D/E` → prefix	`-*A+BC/DE`
`A*(B+C)-D/(A*K)+L` → prefix	`+-*A+BC/D*AKL`
`(A-B/C)*(A/K-L)` → prefix (the notes' 3-step method)	`*-A/BC-/AKL`
`9 3 4 * 8 + 4 / -` (postfix value)	**4** — it is `9 - ((3*4)+8)/4`
4-disc Hanoi	**15** moves
`n = 5` tridiagonal non-zeros	**13**
Bubble on `78 23 45 8 32 36`	5 passes, swaps `5, 3, 1, 0, 0`
Merge `2 9 11 13 57` with `1 3 17 25 90`	`1 2 3 9 11 13 17 25 57 90`
Radix on the 14 keys	sorted in **2** passes
`h(169)`, `h(130)` with `m = 7`	**1** and **4**
Mid-square of `1632`, `1739`, `3123`	**34**, **41**, **31**
Folding of `76321` into 2-digit parts	**09** (`76 + 32 + 1 = 109 mod 100`)
Nine keys, `h(k)=k mod 10`, linear probing	15 probes, ASL **1.67**
Same keys, quadratic probing	13 probes, ASL **1.44**
Same keys, double hashing at `m = 10`	**541 fails** — `m` not prime
Same keys, chaining	max chain **2**, no overflow
Binary search, `n = 10⁶`	**20** comparisons

9.4 The ten mistakes that cost marks

1. Writing "binary search is `O(n)`" or applying it to an **unsorted** array — it returns a wrong answer, not a slow one (§7.12).
2. Forgetting the swap-flag in bubble sort, so you claim the best case is `O(n²)`.
3. Saying selection sort is stable. It is not — show the swap that jumps `a` over `c` (§7.9).
4. Saying quick sort is always `O(n log n)`. Worst case is `O(n²)`, on sorted input with a fixed pivot.
5. Calling merge sort in-place. It needs `O(n)` auxiliary space; that is its only weakness.
6. Confusing `3n − 2` (tridiagonal) with `n(n+1)/2` (triangular), or forgetting which of `i < j` / `i > j` is the *lower* triangle.
7. Writing the 2-D row-major formula with `M` instead of `N` (the number of **columns** multiplies the row index).
8. Postfix/prefix conversion: forgetting that `^` is right-associative, or not mirroring the brackets when using the reverse trick for prefix (§4.10).
9. Deleting a node from a BST by forgetting the three cases (leaf / one child / two children), or replacing with the predecessor when the notes ask for the successor.
10. Priority queue: promoting by *arrival* instead of *priority*, and not keeping FCFS among equal priorities (§4.16).

---

10. Errata in the notes themselves

These are real defects in the five unit PDFs. Each was found by executing the material, so you can quote the notes and still write the correct answer.

#	Where	What the notes say	The correction
1	U4, mid-square	take the "fourth and **fifth** digits counting from the right"	the printed results **34 / 41 / 31** are produced by the **4th and 3rd** digits from the right — the two middle digits of a 7-digit square. The numbers are right, the wording is not. Write the wording the safe way: "extract the middle digits of `k²`".
2	U4, folding	cuts the key into parts without saying from which end	the notes' value for `76321` is **09**, which only comes from grouping from the **left** (`76 + 32 + 1 = 109`). Grouping from the right gives `21 + 63 + 7 = 91`. Even-length keys hide this; odd-length ones expose it. Say "from the left" explicitly.
3	U5, listing headed "Selection Sort"	the inner loop swaps whenever it finds a smaller element	that is **exchange sort**, not selection sort. Real selection sort finds the minimum index first and swaps **once** per pass. `ds/c/notes_selection_listing.c` reproduces the notes' version; `ds/c/selection_sort.c` is the correct one. If asked "how many swaps in selection sort?", the answer is `n - 1` — which the notes' listing does not achieve.
4	U5, page headed "Binary Search"	the code under that heading starts with `void bubbleSort(int *arr, int size)`	not an error in logic but a trap in reading: the notes sort with bubble sort and *then* call `binSearch(arr, 0, size-1, data)`. That call to `bubbleSort` is load-bearing — remove it and the search is wrong. Quote it as the demonstration that binary search needs sorted input.
5	U5, search flags	binary-search pseudo-code sets `flag ← 1` at step 1 and clears it on success; the linear-search listing sets `flag = 1` on success and tests `if (flag == 0)`	the two conventions are **opposite**. Fine on their own, fatal if you mix them in one answer sheet. State the convention you use before using it.
6	U5, linear-search listing	`for (int i = 0; i < 5; i++)` with no `break`	the bound `5` is hard-coded instead of the array's `size`, and with no `break` the loop keeps scanning after a hit, so it prints every match and costs `n` comparisons even when the target is at index 0. Fix both when you write it.
7	U5, binary-search step 2	written as `while (low < high)`	the C listing on the next page says `while (low <= high)`, and it must be `<=` — with `<` a single-element interval is never examined, so a target sitting alone in the last interval is reported missing. Write `low <= high`.
8	U2, stack operations	the third operation is spelled **PEEP**	the standard name is **PEEK** (read the top without removing it). Keep the notes' definition, quietly use the correct spelling.
9	U3, height / depth	the two are described loosely and near-interchangeably	**depth** is measured from the **root down** to a node; **height** is measured from a node **down to its deepest leaf**; the tree's height = the root's height = its maximum depth. A question asking "the height of node X" is not asking for X's depth.
10	U4, page count	Unit 4 is 6 scanned pages, 4.3% of the material	it is the thinnest unit but hashing is a guaranteed 4- or 10-mark question. §6 of this key expands it with the worked collision comparison the notes do not contain. Do not revise Unit 4 from the notes alone.
11	U1, "Updation"	the nine operations list "Updation"	a non-standard word for **Update / Modification**. Harmless if you write it as the notes do; just know the standard term.
12	Whole folder	—	the folder contains **no previous-year question paper** for 2BTCO301, so the exam pattern used in §2 is borrowed from the sibling Semester-3 papers (Q.1 = 10 × 2 marks with no choice; Q.2–Q.5 = 4 + 6 + 10 with an internal choice on the 10-mark question). Treat the pattern as a strong guess, not a confirmed fact.

#	Where	What the notes say	The correction
13	U2, the 3-step prefix trick	"reverse the expression, interchange the brackets, find the postfix, reverse it"	followed **literally** the trick is only safe when no two operators of the same precedence sit next to each other. On `A-B+C` it returns `-A+BC`, which means `A - (B + C)`; the correct prefix of `(A-B)+C` is `+-ABC`. The reason is that reversing flips the left-associative order, so the middle pass must treat every operator as **right-associative** (pop only on strictly greater precedence). `infix_to_prefix` in `ds/verify_ds.py` does exactly that, and both `A*(B+C)-D/(A*K)+L` and `(A-B/C)*(A/K-L)` then come out right. On the exam, write the trick as the notes state it — but check your answer by converting back.

11. How to use this key in the last week

1. **Day 1–2 — Unit 2 and Unit 5 first.** Together they are 76% of the note volume and almost every 10-mark question. Trace bubble, selection and the `A*(B+C)-D/E` conversion by hand on paper, not by reading.
2. **Day 3 — Unit 3 trees**, then Unit 4 hashing. For trees, the exam almost always gives a preorder and an inorder and asks for the tree plus the postorder. Practise §5.6's reconstruction until it is muscle memory, and remember preorder + postorder cannot reconstruct a tree uniquely.
3. **Day 4 — Unit 1**, which is pure definition: data types, linear vs non-linear, the nine operations, time/space complexity with best/average/worst. It is 20 marks of free marks in Q.1.
4. **Every day — one program from §8**, compiled and run. A written program is the highest-risk item in a data-structures paper: the logic is short but the syntax loses marks.
5. **Morning of the exam — §9 only.** The formulas, the numbers table, the ten mistakes.

If you can produce the bubble pass table, the postfix of `A*(B+C)-D/E`, the address of `A[3][4]`, the triplet form of a sparse matrix, a BST after seven inserts, one hashing table under linear probing, and the binary-search trace with its `⌊log₂ n⌋ + 1` bound, you have covered the great majority of what this unit can ask.